wirelessLAN PROFESSIONALS

WLAN PROS TOOLBOX

Field Manual

The complete reference to every tool in the Toolbox, drawn from the app's own help text.

Compiled 2026-06-05 · covers 101 tools · app v1.1.0

This field manual documents every tool in the WLAN Pros Toolbox, drawn directly from the help text that ships inside the app. Each entry states what the tool does, why it is in the kit, how to drive it, the inputs it takes, the formula or method behind it where one applies, a worked example where one helps, and the field notes that keep you out of trouble. Tools are grouped and ordered the same way they appear in the app, so you can navigate the manual and the Toolbox the same way. Every figure and method is the one the app actually runs.

Contents

Grouped and ordered to match the app. Each category lists its tool count; Calculators and Quick Reference list their sections.

CATEGORY

Test Network

4 tools

Live Wi-Fi and internet diagnostics. These tools read the device's real connection, namely the associated AP, link rates, signal, and throughput, and answer the everyday question of whether a slowdown is the Wi-Fi or the internet.

Test My Connection

Answer the everyday question "is the slowdown my Wi-Fi or my internet?" in plain English, and tell the user what to say to support.

WHY IT'S HERE

The consumer-facing front door. Same backends and verdict engine as the pro Wi-Fi vs Internet tool, re-skinned for a non-technical user. Reach for it when you want a one-tap answer plus vetted self-help steps, not engineer numbers.

HOW TO USE

1. Open the tool and tap Run.
2. On iOS, install the companion Shortcut first if prompted; the Wi-Fi link read comes from it (see Wi-Fi Information). Without it, the tool still measures the internet and degrades honestly.
3. On macOS, granting Location is optional; it only affects whether the SSID name shows. The verdict never needs Location (it is rate-based). (test_my_connection_screen.dart:188)
4. Read the two status chips (Wi-Fi / Internet) and the headline.

FORMULA OR METHOD

> It runs two things in one pass and feeds both into the shared verdict engine: a connected-link read via WifiInfoSourceResolver → MacWifiInfoAdapter (macOS CoreWLAN) or WiFiDetailsBridge → ConnectedAp.fromWifiDetails (iOS Shortcut) (test_my_connection_screen.dart:11-13, 142-208); a full net_quality run via OwnEngineQualityClient.forHost('one.one.one.one') (test_my_connection_screen.dart:154). It translates the net_quality grades into the engine's internet-health flag: GOOD only when download AND upload AND latency AND loss all grade good or excellent; otherwise marginal (test_my_connection_screen.dart:308-324). The engine's five engineer verdicts are collapsed into four consumer outcomes plus a two-axis chip status (Wi-Fi / Internet each "Fine", "Slow", or "Couldn't check") (consumer_verdict.dart:33-96, 159-249). Outcomes: A "Looks like your Wi-Fi"; A-lead "Mostly your Wi-Fi"; B "Looks like your Internet"; C "Both look fine"; D1 "Couldn't check everything" (internet measured, Wi-Fi not); D2 "Couldn't complete the check" (neither measured) (consumer_verdict.dart:164-247).

FIELD NOTES

- Platform differences: macOS reads the link via CoreWLAN (Tx rate, RSSI/SNR, no Rx). iOS reads it via the Shortcut (Tx and Rx). If the link can't be read (wired, or iOS without the Shortcut), it falls to the D1/D2 honest path; it does not guess a side. (consumer_verdict.dart:217-247)
- The headline is a hedge by design ("Looks like...", not "your Wi-Fi is broken"). The two chips teach the model that Wi-Fi and internet are two separate things. "Both look fine" is the most useful real-world answer; it points the user at the specific app/site instead of the connection.
- The verdict copy is deliberately non-diagnostic for a layperson. The underlying numbers and bands are the same as Wi-Fi vs Internet; if you want the numbers, use that tool.
- How it picks a side: it compares how much internet speed you're actually getting against how much your Wi-Fi link could realistically carry. When the internet measures fine on its own, it says both look fine; otherwise it leans toward Wi-Fi or internet depending on which one has the headroom to spare.
- D2 ("Make sure you're on Wi-Fi and try again") is the honest answer when nothing measured, never a fake zero.

Source: `test_my_connection_screen.dart:11-208 / consumer_verdict.dart:33-249`

Network Quality

A one-shot transport-quality measurement covering latency, jitter, loss, download, upload, and responsiveness, plus a reachability check against popular sites. Each dimension is graded on its own; there is deliberately no single composite "score".

WHY IT'S HERE

When you want to characterize a connection's transport behavior the way Apple's networkQuality or an Orb does, but as the app's own honest measurement. Reach for it to see whether the internet path is healthy across multiple axes, not just "how many Mbps".

HOW TO USE

1. Open the tool. A live latency trend starts sampling immediately (every 30s) while the screen is open. (net_quality_screen.dart:109-112, 559)
2. Tap "Run test" for the full one-shot measurement (download/upload/responsiveness run only on a full run). (net_quality_screen.dart:497-503)
3. Read the six graded rows and the popular-sites reachability table.

FORMULA OR METHOD

> All this app's own engine (packages/net_quality). Latency / jitter / loss: 10 sequential TCP-connect RTTs to one.one.one.one:443 (not ICMP — the sandbox blocks raw sockets). Jitter is RFC-3550-style mean deviation between consecutive samples; loss is failed-connects ÷ attempts × 100. With zero successful samples, latency and jitter report "Unavailable" but loss is a real 100% (latency_probe.dart:58-163; own_engine_quality_client.dart:94-150). Download: parallel-summed, multi-CDN. Two concurrent streams in one shared window, each against a different endpoint (Cloudflare speed.cloudflare.com/__down, OVH proof.ovh.net, Cachefly), summing bytes over the wall-clock window (throughput_probe.dart:82-169, 245-293). ~25 MB/stream, 10s window cap. If every stream fails, it raises an honest "couldn't measure" — never a fake 0 Mbps (throughput_probe.dart:281-286). Upload: single stream with multi-CDN fallback (only Cloudflare __up is a verified large-POST sink — honest single-stream, not faked parallelism). A non-2xx or empty transfer is an honest failure, not 0 (throughput_probe.dart:93-96, 410-451). Responsiveness (RPM): a simplified single-flow loaded-latency estimate inspired by RFC 9097 / Apple networkQuality, NOT the full multi-flow RPM standard. It samples loaded RTT while a download flow runs, then RPM = 60000 / loadedAvgMs (responsiveness_probe.dart:39-85; own_engine_quality_client.dart:179-197). Reachability: TCP-connect (port 443) to 12 well-known hosts (popular_sites.dart:27-45; reachability_probe.dart:37-88). Grade bands (scoring.dart): Latency ms — Excellent <20, Good <50, Fair <100, Poor ≥100 (grounded in ITU-T G.114, our cut points). Jitter ms — <5/<15/<30. Loss % — 0/<1/<2.5. Responsiveness RPM — ≥1000/≥500/≥100. Download Mbps — ≥100/≥25/≥5 (explicitly a heuristic). Upload Mbps — ≥20/≥5/≥1 (heuristic) (scoring.dart:20-88).

FIELD NOTES

- Platform differences: runs on macOS, Windows, Linux, Android, iOS over dart:io sockets/HTTP. On web it routes to the download-the-app fallback (no sockets). (net_quality_screen.dart:17-24, 286-292)
- Read each grade word on its own; a connection can be Excellent on latency and Poor on upload at the same time, which is the point. The download/upload Mbps grades are "good enough for a household" heuristics, not standards. RPM is directional only.
- Latency uses TCP handshake RTT, so it includes the full SYN/SYN-ACK round trip to a real host (a faithful proxy, slightly higher than ICMP).
- The on-screen footnote states plainly: "these are this app's own measurements, not an Orb or Ookla score." Download is summed-parallel (so it reflects aggregate link capacity, not a single flow). A failed measurement is "Unavailable" with a note, never 0.

Source: `net_quality_screen.dart / packages/net_quality (scoring.dart:20-88, latency_probe.dart:58-163, throughput_probe.dart:82-451, responsiveness_probe.dart:39-85)`

Wi-Fi Information

Show the live connected-AP link details: SSID, BSSID, RSSI, noise, SNR, Tx/Rx rate, channel, width, band, 802.11 standard.

WHY IT'S HERE

The "what is my radio actually doing right now" read. Reach for it to confirm band/channel/width, check signal and SNR against design targets, or sanity-check a client's negotiated rate.

HOW TO USE

1. macOS: open the tool; it pulls a CoreWLAN snapshot. Tap Refresh to re-read. SSID and BSSID are gated behind macOS Location Services; tap Grant (or open the Location settings pane) to reveal them. The radio metrics do not need Location. (wifi_info_screen.dart:9-12; wifi_info_service.dart:184, 219-240)
2. iOS: this is live streaming only. Install the combined "WLAN Pros Live" companion Shortcut (one-time, via the iCloud link), then tap Start. The Shortcut loops, harvesting the connected AP each cycle via the stock "Get Network Details" action and handing JSON back to the app; Stop freezes the last values. There is no one-tap snapshot on iOS. (wifi_info_screen.dart:13-18; wifi_live_shortcuts_config.dart:18-42; wifi_details_bridge.dart:115-156)

FORMULA OR METHOD

> All sources normalize into one ConnectedAp model (connected_ap.dart). macOS comes from CoreWLAN via the com.wlanpros.toolbox/wifi_info native channel (wifi_info_service.dart:151-213); iOS comes from the Shortcut JSON parsed by WiFiDetails (case-insensitive keys, tolerant numeric parse) (wifi_details.dart:151-204). In iOS Live mode, RSSI and SNR get hard grades against Keith-reviewed bands; Tx/Rx rates get a trend (Rising/Falling/Steady) rather than a hard grade, because a "good" data rate is entirely relative to band/width/MCS (wifi_grading.dart:9-23, 38-84). RSSI bands (dBm): Excellent ≥-59 (>-60), Good ≥-67, Fair ≥-72, Poor below. SNR bands (dB): Excellent ≥36 (>35), Good ≥25, Fair ≥15, Poor below (wifi_grading.dart:50-77).

FIELD NOTES

- macOS (CoreWLAN): exposes SSID/BSSID (Location-gated), RSSI, noise, SNR (reported directly), Tx rate, PHY mode → standard label, channel, channel width, band (reported), country code, interface name, hardware MAC. It does NOT expose the Rx rate or Tx power (public CoreWLAN limitation); those render "Not exposed by macOS CoreWLAN", never estimated. (wifi_info_service.dart:16-20, 64-65; connected_ap.dart:120-149)
- iOS (Shortcut): exposes SSID, BSSID, channel, RSSI, noise, standard, Rx rate AND Tx rate. SNR is derived app-side (rssi – noise) and band is derived from the channel number (both labeled "derived"). The harvest does NOT return channel width, so width renders "Not reported by iOS", never fabricated. Channel→band derivation: 1 to 14 → 2.4 GHz, 36 to 177 → 5 GHz, 181 to 233 → 6 GHz; the ambiguous low 6 GHz range (1 to 93) is read as 2.4/5 GHz and never silently claimed as 6 GHz. (wifi_details.dart:19-23, 42-66, 116-132; connected_ap.dart:151-182)
- Android / Windows: honest "coming in a later update" state (clean seam, not built). Web: download-the-app fallback. (wifi_info_adapter.dart:36-68)
- RSSI and SNR carry hard grades; read the rate as a trend, not a pass/fail. The standard label combines the 802.11 designation and Wi-Fi generation (e.g. "802.11be (Wi-Fi 7)"; 802.11ax on 6 GHz shows Wi-Fi 6E). Anything marked "derived" was computed, not read from the radio. (connected_ap.dart:199-215)
- macOS Tx-only and iOS no-width are real platform ceilings, not bugs. On macOS, if the SSID is blank, it's almost always a missing Location grant, not a hidden network. The macOS read has a 5s hang-safety: a stalled CoreWLAN read surfaces an honest "No Wi-Fi reading" rather than freezing. (wifi_info_adapter.dart:143-184)

Source: `wifi_info_screen.dart / wifi_info_service.dart / wifi_details.dart / connected_ap.dart / wifi_grading.dart:50-77`

Cellular Information

Show the iPhone's mobile-network details: carrier, radio technology, signal bars, country code, roaming.

WHY IT'S HERE

A quick read of what the cellular radio is doing, useful when comparing Wi-Fi offload behavior or confirming a device fell back to LTE/5G.

HOW TO USE

1. iOS only. Install the same combined "WLAN Pros Live" companion Shortcut, then Start; the Shortcut harvests cellular details via "Get Network Details" and hands them over the App Group bridge. (cellular_info.dart:16-18; cellular_info_adapter.dart:8-17)

FORMULA OR METHOD

> Parses the Shortcut JSON into CellularInfo (case-insensitive keys, with whitespace-trim tolerance after a real on-device bug) (cellular_info.dart:99-173). Fields: carrier name, radio technology (mapped from raw CTRadioAccessTechnology* constants to friendly labels like "5G (NSA)", "LTE"; unknown values pass through, never blanked), signal bars (coarse 0-4 status-bar indicator), country code, roaming bool (cellular_info.dart:36-66, 175-207).

FIELD NOTES

- Platform differences: iOS is the only source. There is deliberately no native CoreTelephony path: CTCarrier is deprecated since iOS 16.4 and returns placeholder junk, and cellular signal strength (RSRP/RSRQ/dBm) is private-API-only and an App Store rejection. So data comes only via the Shortcut. macOS (no radio), Android, Windows show an honest "not available on this platform"; web shows the download fallback. (cellular_info.dart:12-25; cellular_info_adapter.dart:28-58)
- Signal bars are bars (0 to 4), never relabeled dBm/RSRP/RSRQ; the app does not have a raw signal value and must not imply it does. Bars are clamped to 0 to 4; an out-of-range value is never trusted. (cellular_info.dart:20-23, 55-58, 127-137)
- This is the one tool where iOS exposes more than a native app could (the Shortcut runs in Apple's privacy context). A missing field renders "Unavailable", never a fabricated value.

Source: `cellular_info.dart:12-207 / cellular_info_adapter.dart:8-58`

CATEGORY

Networking Tools

22 tools

Socket, lookup, and scan utilities for working a network from the device in hand. Ping, traceroute, port and host discovery, DNS and registry lookups, and packet-level senders and inspectors.

Device Info

Show the device's own system facts: model (marketing name plus the raw identifier), total physical memory (RAM), system uptime since the last boot, and the cellular IP address where the device has a cellular interface.

WHY IT'S HERE

The companion to Interface Information: that tool answers "what's my IP, gateway, and Wi-Fi link"; this one answers "what device is this, how much RAM, how long since boot, and what's my cellular IP". Reach for it to confirm the hardware model or to read the cellular-side address that the Wi-Fi-centric interface view doesn't surface.

HOW TO USE

1. Open the tool; it reads a snapshot. Each field a platform can't provide renders its honest unavailable state rather than a fabricated value.
2. Tap Refresh in the top bar to re-read (uptime advances; the rest is stable).
3. Use Copy to put the whole snapshot on the clipboard as labeled text.

FORMULA OR METHOD

```
Model and total memory come from the device_info_plus package (BSD-3): on iOS, modelName (the package maps utsname.machine — e.g. iPhone16,2 — to a marketing name) plus physicalRamSize; on macOS, modelName/model plus memorySize. Uptime comes from a tiny native MethodChannel (com.wlanpros.toolbox/system_info → systemUptime) reading ProcessInfo.processInfo.systemUptime — no package exposes it. The cellular IP comes from dart:io NetworkInterface.list, matching the conventional iOS cellular interface name pdp_ip0 (device_info_service.dart).
```

FIELD NOTES

- Cellular IP uses a heuristic: Apple does not treat interface names as a stable API, so detection matches the conventional iOS cellular name pdp_ip0. "No cellular interface" is the normal, honest result on a Wi-Fi-only iPhone, in airplane mode, or on a Mac (which has no cellular interface).
- Total memory is shown in binary units (an 8 GiB device reads "8 GB") to match how RAM is physically sized; the label stays the familiar GB/MB.
- Uptime is seconds since boot, formatted as "3d 4h 12m"; it always shows at least minutes (a just-booted device reads "0m").
- Nulls are honest "not available", never 0 or "". On Android, model is shown but total RAM is not surfaced by the package, so it reads unavailable.

Source: `device_info_service.dart; device_info_format.dart; system_uptime_bridge.dart; macos/Runner/SystemInfoChannel.swift; ios/Runner/SystemInfoChannel.swift`

Inspector (HTTP Header)

Issue a HEAD or GET, follow and record the redirect chain hop-by-hop, and return the final status plus all response headers.

WHY IT'S HERE

Shows the full 301→302→200 story, not just the destination. Reach for it to debug redirects, security headers, caching, or CDN behavior.

HOW TO USE

1. Enter a URL (bare host assumes https://), pick HEAD (default) or GET, inspect. Each hop is shown with its status, Location, and headers.

FORMULA OR METHOD

> Sets followRedirects = false and follows the chain itself, recording one hop per response until a non-redirect status or the 10-redirect cap (http_header_service.dart:5-13, 198-273). HEAD→GET fallback: if HEAD returns 405 (and the caller didn't demand GET), it transparently retries that hop with GET and notes the fallback (http_header_service.dart:15-20, 224-238). Relative Location values are resolved per RFC 7231 (http_header_service.dart:250-261, 330-343).

FIELD NOTES

- Platform differences: iOS App Transport Security blocks cleartext http://, and the app deliberately does not add a blanket ATS exception (that would weaken every request). So an http:// target fails at the socket layer on iOS, and the tool surfaces a specific message ("On iOS, cleartext HTTP is blocked by ATS, try the https:// URL") rather than a generic failure (http_header_service.dart:22-27, 352-359). Otherwise identical on native; gated off on web.
- The hop chain reads top (first request) to bottom (final response). headFellBackToGet and redirectLimitHit flags are surfaced. Header names are title-cased and sorted.
- Bodies are drained and discarded; this is a header inspector, not a fetcher. On iOS, use the https:// URL.

Source: `http_header_service.dart:5-359`

Inspector (SSL/TLS)

Connect to host:port over TLS and report the server certificate as inspectable data, including expired, self-signed, and name-mismatch certs.

WHY IT'S HERE

A field cert inspector. Reach for it to read validity, SANs, fingerprints, key size, and the issuer on any TLS service, especially broken ones.

HOW TO USE

1. Enter a host (URLs are accepted and stripped to the host), optional port (443 default), inspect.

FORMULA OR METHOD

```
SecureSocket.connect with onBadCertificate: (cert) => true — it accepts any certificate at the socket layer so an expired/self-signed/mismatched cert is still captured and shown; the validity verdict is computed from the cert dates, not thrown as an error (ssl_inspect_service.dart:1-12, 251-265). A bad cert is a successful inspection, not a failure (ssl_inspect_service.dart:171-174). Field coverage is split: dart:io X509Certificate gives PEM/DER, subject/issuer DN, notBefore/notAfter, SHA-1; basic_utils X509Utils re-parses the PEM to recover structured DN, SAN list, serial, signature algorithm, public-key algorithm + bits, and SHA-256 (ssl_inspect_service.dart:13-34, 360-457). Validity is computed against "now": valid / expired / not-yet-valid + days-to-expiry (ssl_inspect_service.dart:46-100).
```

FIELD NOTES

- Platform differences: same on every native platform. Gated off on web. (ssl_inspect_service.dart:35-36)
- The validity state is an icon + text (not color alone). Two honest limits are stated on-screen, not faked: (1) ALPN ≠ TLS version/cipher, since dart:io exposes only the ALPN result, not the negotiated TLS version or cipher suite, so those are reported "not exposed by the platform", never invented (ssl_inspect_service.dart:23-29, 211-214). (2) Leaf only, since dart:io hands over only the leaf certificate, not the intermediate/root chain it validated against (ssl_inspect_service.dart:30-34, 223-227).
- Connection problems (DNS, refused, timeout) are failures; an invalid cert is not. Don't read the ALPN line as the TLS version; the tool deliberately separates them.

Source: `ssl_inspect_service.dart:1-457`

Interface Information

Show the device's own network state: per-interface IPv4/IPv6 addresses, interface name and inferred type, plus gateway, DNS, Wi-Fi SSID/BSSID, and the device's primary IP where the platform exposes them.

WHY IT'S HERE

The "what's my IP, gateway, and link" foundation. Reach for it first when you need the device's own address (e.g. before a ping/sweep) or to confirm which interface is active.

HOW TO USE

1. Open the tool; it reads a snapshot. Each field that a platform can't provide returns null and renders "Not available on this platform".

FORMULA OR METHOD

> Built on dart:io NetworkInterface.list for the address/interface table, plus network_info_plus for Wi-Fi-link details (SSID, BSSID, gateway, subnet mask, Wi-Fi IPv4/IPv6) that dart:io doesn't expose (interface_info_service.dart:23-25, 144-191). Each sub-read is independently guarded so one denied call (e.g. SSID) never blanks the whole screen (interface_info_service.dart:129-142, 193-202). Interface kind (Wi-Fi/Ethernet/Cellular/Loopback/VPN/Other) is a heuristic from the OS interface name — e.g. macOS en0 is guessed Wi-Fi, en1+ Ethernet; explicitly conservative (interface_info_service.dart:227-262). primaryIPv4 prefers the Wi-Fi link IP, else the first non-loopback IPv4 (interface_info_service.dart:96-106).

FIELD NOTES

- Platform differences: SSID/BSSID/gateway depend on network_info_plus and OS permission state. iOS needs the wifi-info entitlement + Location for SSID/BSSID; macOS/Android vary. Web is gated off. SSID is cleaned of wrapping quotes and the <unknown ssid> placeholder. (interface_info_service.dart:60-79, 204-213)
- The interface "type" is a name-based guess; on macOS the en0/en1 split is heuristic, so trust the addresses over the label on unusual hardware.
- Nulls are honest "not available", never 0 or "". Gateway/DNS exposure is platform-and-permission dependent.

Source: `interface_info_service.dart:23-262`

IP Geolocation

Country, region, city, coordinates, timezone, ISP/org, and ASN for an IP (or your own public IP).

WHY IT'S HERE

Quick geo + ownership context for an address. Reach for it to see where an IP resolves and who runs it.

HOW TO USE

1. Enter an IP or hostname, or leave blank to locate your own public IP, look up. A copyable "lat,long" and an OpenStreetMap URL are offered.

FORMULA OR METHOD

> `Queries ipwho.is (free, no key, HTTPS — chosen because it's keyless AND returns ASN + timezone in one response; ip-api.com is HTTP-only and would trip iOS ATS, so it's explicitly not used) (ip_geo_service.dart:1-27). An empty query hits https://ipwho.is/ (your IP); otherwise https://ipwho.is/{ip} (ip_geo_service.dart:157-189). A cheap client-side sanity check rejects obvious junk before spending a round-trip; ipwho.is's in-band {"success": false} is mapped to a precise "check your input" or rate-limited state, not read as OK (ip_geo_service.dart:166-218, 253-268).`

FIELD NOTES

- Platform differences: JsonHttpClient → dart:io, native-only; web gated to the download fallback (ipwho.is CORS unverified). (ip_geo_service.dart:23-25)
- Coordinates are shown as selectable mono data plus a copyable pair and an external maps link (no embedded interactive map; that's future) (ip_geo_service.dart:18-21, 125-147). Every field is nullable → missing data renders "Not available".
- Geolocation accuracy is the provider's, especially for mobile/CGNAT IPs. A rate-limit response is surfaced explicitly ("wait a minute and try again").

Source: `ip_geo_service.dart:1-268`

IP Subnetting (IPv4)

Computes the full IPv4 subnet breakdown (network, broadcast, netmask, wildcard, first/last usable host, total addresses, and usable host count) from an address plus a CIDR prefix or a dotted mask.

WHY IT'S HERE

CIDR math you'd otherwise do on paper or in your head at a whiteboard. Reach for it to confirm a network/broadcast boundary, a usable-host range, or a mask↔prefix conversion.

HOW TO USE

1. Enter an IPv4 address (e.g. 10.20.0.0). You can append the prefix inline as 10.20.0.0/22; an inline /prefix wins and the second field is ignored.
2. Otherwise enter the prefix or mask in the second field: a CIDR prefix (22 or /22) or a dotted mask (255.255.252.0).
3. The breakdown recomputes live on every valid keystroke. A host inside the subnet (e.g. 10.20.0.37/22) reports the same network as the base address.
4. Malformed input shows an inline "Check your input" card with a specific message rather than a wrong answer.
5. Use the Copy action in the app bar to copy the breakdown as a labeled text block.

INPUTS

INPUT	UNIT	RANGE
IPv4 address	dotted-decimal, optionally with inline /prefix	four octets each 0-255; inline /prefix 0-32
Prefix or mask	CIDR prefix or dotted mask	prefix 0-32, or a contiguous-ones dotted mask (e.g. 255.255.252.0); ignored when the address carries an inline /prefix

FORMULA OR METHOD

> Pure-Dart 32-bit integer math on the masked network base (subnet_calc_service.dart:93-177). mask = (0xFFFFFFFF << (32 − prefix)) & 0xFFFFFFFF (:242-246); network = addr & mask (:134); broadcast = network | (~mask & 0xFFFFFFFF) (:135); wildcard = ~mask & 0xFFFFFFFF (:136); total = 2^(32 − prefix), with /0 = 2^32 (:137-139). Usable hosts by prefix: /0-/30 → total − 2 with first = network+1, last = broadcast−1 (:157-162); /31 → RFC 3021 point-to-point, usable = 2, no broadcast, both addresses are hosts (:151-156); /32 → single host route, usable = 1, first = last = the address, no broadcast (:146-150). A dotted mask is converted to a prefix only if it's a contiguous run of 1 bits (prefixFromMask, :218-237). Address parsing is strict: exactly four octets, each 0-255 (:199-211).

EXAMPLE

> 10.20.0.0/22 → netmask 255.255.252.0, wildcard 0.0.3.255, network 10.20.0.0, broadcast 10.20.3.255, first host 10.20.0.1, last host 10.20.3.254, total 1024, usable 1022. (This is the screen's seeded default.)

FIELD NOTES

- /31 (RFC 3021) is a point-to-point link: there is no network/broadcast reservation, so both addresses are usable hosts (usable = 2). The screen annotates this.
- /32 is a single-host route: one address, no range, no broadcast (usable = 1).
- A host address inside the block reports the subnet's network, not the host; all values derive from the masked base.
- A dotted mask must be a valid contiguous-ones mask; 255.0.255.0 is rejected as malformed.
- No network I/O, just pure math, so it runs on every platform including web.

Source: `subnet_calc_service.dart:93-177`

IP Subnetting (IPv6)

From an IPv6 address and prefix length, derives the expanded and compressed forms, network address, first/last address in the prefix, host count, and RFC address type.

WHY IT'S HERE

IPv6 subnetting and address-type identification during network design and troubleshooting, where 128-bit math is error-prone by hand.

HOW TO USE

1. Enter an IPv6 address (default 2001:db8::1).
2. Enter the prefix length (default 32).
3. Read expanded form, compressed form, network/prefix, first and last address, host count, and address type.

INPUTS

INPUT	UNIT	RANGE
IPv6 address	`any valid IPv6 literal`	`exactly one :: run allowed; invalid format → inline error`
Prefix	`integer`	`0 to 128 (out of range → error)`

FORMULA OR METHOD

```
Expand :: to full 8-group / 4-hex-digit form, rejecting more than one :: (:99-133). Pack to a 128-bit BigInt (toBigInt). mask = ((1 << prefix) - 1) << (128 - prefix) (0 when prefix 0); network = addr & mask; last = network | (~mask & 128-bit-mask) (:275-280). Compress to canonical :: by collapsing the longest run of all-zero groups (compressIPv6). Host count (:229-234): host bits = 128 - prefix; > 63 → "More than 2⁶³"; 0 → "1 address"; else 2^hostBits with grouping. Address type by prefix match (detectIPv6Type): :: unspecified, ::1 loopback, fe80 link-local, fc/fd unique-local, ff multicast, 2002 6to4, ::ffff: IPv4-mapped, 2001:db8 documentation, else global unicast. (ipv6_subnet_screen.dart:94-295)
```

EXAMPLE

2001:db8::1 / 64 → expanded 2001:0db8:0000:0000:0000:0000:0000:0001; compressed 2001:db8::1; network 2001:db8::/64; host bits = 64 (> 63) → host count shows "More than 2^{63}"; type = Documentation (2001:db8::/32).

FIELD NOTES

- Host counts above 2^{63} are reported qualitatively ("More than 2^{63}") rather than as a full number.
- The "first address" is the network address itself (IPv6 has no broadcast and does not reserve the all-zeros host as IPv4 does).
- Address-type detection is prefix-pattern based and covers the common RFC ranges, not every reserved block.
- MANUAL-VS-CATALOG NOTE: the manual labels this tool's category as "Networking (calculator screen; grouped under Networking in the catalog, tool_catalog.dart:324)". In the current 4-category catalog this id sits under the "Networking Tools" category (id networking).

Source: `ipv6_subnet_screen.dart:94-295`

Lookup (ARP/NDP)

Discover local-network neighbors: IP, and MAC where the platform exposes it.

WHY IT'S HERE

A neighbor read of the local segment. Reach for it to map IP↔MAC on platforms that can, or just to list responders.

HOW TO USE

1. Open and run; it derives the local subnet and sweeps it, attaching cached MACs where available.

FORMULA OR METHOD

```
Active discovery — derive the local /24 (or real prefix), TCP-connect-probe each host
(curated LAN ports 80/443/22/445/139/53/8080, refused RST counts as up), list the
responders, and on Linux/Android read /proc/net/arp to attach the real cached MAC
(arp_ndp_service.dart:1-34, 130-132, 289-322). No subprocess (arp -a is sandbox-blocked).
Safety cap refuses anything larger than a /22 (1022 hosts) (arp_ndp_service.dart:181-198).
```

FIELD NOTES

- Platform matrix (honest, in the source): Android / Linux, active sweep with MAC from /proc/net/arp (arp_ndp_service.dart:135-147). macOS / Windows, active sweep, no MAC (no readable ARP file; arp -a/GetIpNetTable out of scope), so MAC renders "Not exposed on this platform" (arp_ndp_service.dart:8-16, 144-147). iOS, unavailable; neighbor tables aren't accessible to third-party apps (arp_ndp_service.dart:140-141). Web, download fallback.
- A null MAC means the platform doesn't expose it, never an invented value. Incomplete/all-zero ARP entries are skipped. (arp_ndp_service.dart:28-29, 63-65, 167-169)
- On macOS/Windows you get a responder list without MACs; that's the platform ceiling. For richer enrichment and macOS MAC (via sysctl), see Network Discovery.

Source: `arp_ndp_service.dart:1-322`

Lookup (BGP/ASN)

ASN, holder, announced prefix, registry, and peer/upstream counts for an IP or ASN.

WHY IT'S HERE

Routing-layer context: who announces a prefix, which RIR, how it's connected. Reach for it on upstream/ISP questions.

HOW TO USE

1. Enter an IPv4/IPv6 or an ASN (AS15169, 15169, or as15169), look up.

FORMULA OR METHOD

> `Queries the RIPEstat Data API (free, no key, HTTPS, authoritative — operated by RIPE NCC) (bgp_asn_service.dart:1-26). IP path: network-info (prefix + ASN) then as-overview (holder, type, registry, announced). ASN path: as-overview then asn-neighbours mapped to upstream/peer/downstream counts (bgp_asn_service.dart:199-282). Input is classified IP vs ASN before any call (bgp_asn_service.dart:140-171).`

FIELD NOTES

- Platform differences: built on JsonHttpClient (→ dart:io), so native-only; web is gated to the download fallback (RIPEstat CORS unverified, so no maybe-broken web tool). (bgp_asn_service.dart:22-23)
- Three states: success, empty (API answered cleanly but resolved no ASN, e.g. a private/bogon IP not in the routing table), and failure (bgp_asn_service.dart:123-129). Peer/upstream counts are best-effort enrichment; a neighbours failure leaves them null ("Not available"), not zero (bgp_asn_service.dart:263-266).
- Every field is nullable; a datum the API omits renders "Not available", never fabricated. Neighbour "type" mapping: left→upstream, right→downstream, unknown→peer. (bgp_asn_service.dart:250-257)

Source: `bgp_asn_service.dart:1-282`

Lookup (DNS)

A portable dig/nslookup for the field. Resolve a name as a dig-style all-records view (SOA, NS, A, AAAA, MX, TXT, SRV, CAA at once), as a single record type, or run a one-tap reverse PTR for an IP.

WHY IT'S HERE

Checks records authoritatively rather than from the local cache, and works through captive portals and corporate firewalls. The all-records view is the fast "show me everything" pass; single-type is the focused look; reverse PTR turns an IP back into a hostname without flipping any selector.

HOW TO USE

1. Enter a hostname (or an IP). Leave the mode on All records for the dig-style sweep, then look up.
2. For one record type, switch to Single type and pick A, AAAA, MX, TXT, NS, SOA, PTR, SRV, CAA, or SPF.
3. When the input is an IP, a Reverse lookup (PTR) button appears for a one-tap IP to hostname query.
4. Pick the resolver (Cloudflare default or Google) if one provider is blocked.

FORMULA OR METHOD

> Resolves over DNS-over-HTTPS (DoH) via basic_utils DnsUtils.lookupRecord, an HTTPS GET to a JSON resolver, not raw UDP/53 (dns_lookup_service.dart). DoH is chosen because raw UDP/53 sits behind iOS local-network gating and is often blocked, while DoH rides HTTPS:443 cleanly and needs no extra socket capability. All records mode fans out one DoH query per type (SOA, NS, A, AAAA, MX, TXT, SRV, CAA) concurrently with Future.wait, then groups the answers in dig order; a per-type failure becomes a per-section note, so the records that did resolve still show. Reverse PTR rewrites the input IP to its in-addr.arpa or ip6.arpa name before querying (with a minimal IPv6 literal parser). SPF is not a separate query: it reads TXT and filters for the v=spf1 policy line (RFC 7208). Three result states per query: success, empty (resolved, no records of this type), and failure, kept distinct so the UI shows the right state.

FIELD NOTES

- Records resolve against a public resolver (Cloudflare or Google), not the device configured resolver, which is usually what a pro wants (authoritative, uncached). The device configured DNS servers are shown by Interface Information instead.
- SRV and CAA are parsed into readable fields; an unparseable form is shown raw. Empty is not an error: a name with no MX simply returns the empty state.
- Platform differences: identical on iOS, Android, macOS, and Windows (HTTPS only). Gated off on web per the native-only product decision.
- dig +trace (the root to TLD to authoritative delegation walk) is not built. DoH only reaches recursive resolvers that return the final answer, so a true iterative trace would need raw UDP/53 to named authoritative servers plus a hand-rolled DNS wire codec, which is heavier and less reliable in the field than this tool is meant to be.

Source: `dns_lookup_service.dart, dns_lookup_screen.dart`

MAC Vendor OUI Lookup

Turn a MAC address into its registered vendor, fully offline.

WHY IT'S HERE

Identify a device's manufacturer from its MAC with no network, and, crucially, tell you honestly when a MAC has no real vendor (randomized phones).

HOW TO USE

1. Enter a MAC in any common notation (colon, hyphen, Cisco dot aabb.ccdd.eeff, or no separators; any case), look up.

FORMULA OR METHOD

> Pure-Dart resolver over a bundled IEEE registry table (assets/oui/oui_table.tsv) loaded once and cached — no HTTP, no dart:io (mac_oui_service.dart:1-27, 104-142). It honors the three IEEE block sizes and matches most-specific-first: MA-S (/36, 9 hex), then MA-M (/28, 7 hex), then MA-L (/24, 6 hex), so a /36 sub-allocation names the real sub-assignee, not the /24 parent (mac_oui_service.dart:5-8, 213-228).

FIELD NOTES

- Platform differences: identical everywhere (offline, pure Dart), including web-safe in principle, though it ships in the gated Networking category. No platform exposes more or less here.
- Honesty bits: U/L bit set (0x02) = locally-administered / randomized address (the common case for modern iOS/Android Wi-Fi) → flagged, no vendor invented (a registry hit would be coincidence) (mac_oui_service.dart:10-17, 117-118, 199-211). I/G bit set (0x01) = multicast/group address, not a single NIC → flagged, no vendor (mac_oui_service.dart:120-121). A globally-administered MAC not in the bundled snapshot → the raw 24-bit OUI is shown (e.g. B8:27:EB), never invented (mac_oui_service.dart:257-266).
- A randomized phone MAC correctly returns "no vendor"; that's the right answer, not a miss. The bundled table has a documented retrieval date in its asset header; refresh it to pick up new allocations.
- CATALOG NOTE: manual category is Networking Tools; catalog id is mac-oui-lookup with route /tools/mac-oui.

Source: `mac_oui_service.dart:1-266`

Network Discovery

Find live hosts on the local network and enrich each with name, services, inferred device type, and (where exposed) MAC/vendor.

WHY IT'S HERE

A Fing-style LAN scan. Reach for it for a richer inventory than a bare ping sweep: what each host is, not just that it answered.

HOW TO USE

1. Open and run. The engine seeds the local /24, connect-scans it, reverse-DNS resolves, mDNS-browses, then (macOS) reads the ARP cache for MAC/vendor.

FORMULA OR METHOD

```
Four passes (lan_discovery_engine.dart): 1. Subnet seed — derive the local /24 host list from network_info_plus (:181-199). 2. Connect-scan — bounded-concurrency (64) TCP connect across the /24 × a curated port set, run in a background isolate; streams progress (:201-262). 3. Reverse DNS — InternetAddress.reverse() per discovered host (null when no PTR) (:264-275, 499-510). 4. mDNS browse — in-house native NetServiceBrowser/NetService (Apple Bonjour daemon) over a curated DNS-SD service set (:277-301; mdns_browse.dart:1-51). Then the ARP-cache read (macOS only, via Swift sysctl NET_RT_FLAGS/RTF_LLINFO — never a subprocess) runs after the scan warms the kernel cache, attaching MAC + vendor (:303-335). Finally a pure device-type heuristic runs on each host's open ports + mDNS services (device_type.dart:62-146). Device-type rules (first match wins, most specific first): IPP/LPD/9100 or printing mDNS → Printer; RTSP → Camera/NVR; iOS lockdownd (62078) → iOS device; SMB (445) → Windows/SMB; _sonos/_spotify-connect → Speaker; _googlecast → Media streamer; _airplay/_raop/_companion-link → Apple device; then weak signals 80/443/8080 → Web server, 22 → SSH host; any mDNS → mDNS device; else Unknown (device_type.dart:47-146). MAC→vendor resolves through the full bundled IEEE OUI registry; the resolver owns the honesty contract — null for randomized/local MACs, raw-OUI fallback for unlisted global prefixes, named vendor otherwise (lan_discovery_engine.dart:101-113, 321-334; mac_oui_service.dart:243-266).
```

FIELD NOTES

- Platform differences (the heart of it): MAC + vendor: macOS only (the sysctl ARP read). On iOS/Android a sandboxed app cannot read the ARP table, so MAC/vendor stay null (lan_discovery_engine.dart:303-311). mDNS: iOS + macOS via the native NetServiceBrowser channel. It deliberately does NOT use pure-Dart multicast (iOS 14+ silently drops it without Apple's multicast entitlement) and does NOT use bonsoir (GPL-3.0, incompatible with the closed-source App Store app). Android/other platforms get a clean empty mDNS pass (NsdManager deferred). Service types must be declared in Info.plist NSBonjourServices (mdns_browse.dart:8-51). The connect-scan core is pure-Dart and cross-platform; only mDNS/ARP enrichment are native.
- Device type is a heuristic from ports + mDNS, not a MAC-anchored database; Unknown is a first-class, non-apologetic outcome. The code deliberately does NOT fake an "access point" rule, because the only reliable AP signal is the OUI vendor (MAC), which mobile can't read. (device_type.dart:116-124)
- Any single pass can fail without aborting the run (a failed mDNS browse just means no mDNS enrichment; nothing is faked) (lan_discovery_engine.dart:23-25). On mobile, expect no MAC/vendor and APs to fall through to SSH/Web/Unknown, a documented ceiling.

Source: lan_discovery_engine.dart:23-510 / device_type.dart:47-146 / mdns_browse.dart:1-51

Packet Sender

Send a custom TCP or UDP payload to host:port and read the reply (raw bytes + hex + decoded text).

WHY IT'S HERE

A field "netcat" for poking a service: banner grabs, custom probes, protocol pokes. Reach for it to send a crafted payload and see what comes back.

HOW TO USE

1. Enter host, port, transport (TCP/UDP), and a payload (plain text, or \xNN hex escapes plus \r \n \t \0 \\), send. The reply is shown as hex and decoded text.

FORMULA OR METHOD

> TCP via Socket, UDP via RawDatagramSocket — no raw sockets, no custom ICMP/IP framing (same sandbox wall as ICMP traceroute), so it ships clean everywhere (packet_sender_service.dart:1-22). TCP: connect → send → read until the peer closes or the read goes idle for the timeout, with a hard total cap (timeout×3) so a chatty stream can't run forever (packet_sender_service.dart:11-14, 279-356). UDP: bind ephemeral → send one datagram → wait up to the timeout for a reply; no reply is a first-class non-error outcome (timedOut/isNoReply), not an exception, because UDP has no delivery guarantee (packet_sender_service.dart:16-19, 130-133, 399-472). Errors are typed: DNS failure, refused, unreachable, timeout, invalid input, other — each mapped to a precise message (packet_sender_service.dart:33-51, 358-397).

FIELD NOTES

- Platform differences: identical on every native platform. Gated off on web. (packet_sender_service.dart:21-22)
- A UDP "no reply" is honest and expected for many services; it does not mean failure. The payload parser returns null only on a malformed \x (a clear authoring mistake worth surfacing) (packet_sender_service.dart:162-224).
- Binary replies decode with the Unicode replacement char rather than throwing, so they still render. This is a single-shot send/receive, not an interactive session.

Source: packet_sender_service.dart:1-472

Ping (ICMP)

Real ICMP echo round-trip on mobile: live RTT, min/avg/max, loss.

WHY IT'S HERE

When you specifically want true ICMP echo (the classic ping), available where the platform genuinely supports it.

HOW TO USE

1. Enter a host, run (mobile only). Streams replies with running stats, same UI shape as TCP Ping.

FORMULA OR METHOD

```
Real ICMP echo request/reply via the native backend (dart_ping_ios SimplePing/GBPing on iOS; dart_ping spawning system ping on Android) (icmp_service.dart:1-52, 256-265). The method label is "ICMP echo" — never relabeled from a TCP probe (icmp_service.dart:96-110).
```

FIELD NOTES

- Platform matrix (honest): iOS, real ICMP echo available (icmp_service.dart:297-303). Android, real ICMP echo via system ping, available (icmp_service.dart:299-302). macOS / Windows / Linux desktop: the only ICMP path is spawning /sbin/ping, which the macOS App Sandbox blocks, so it gives an honest "not available in the sandboxed desktop build", and the UI points the user at TCP Ping instead (icmp_service.dart:37-41, 300-303). Web: no sockets → download fallback.
- Where available, this is genuine ICMP RTT/loss, the real thing, not a TCP proxy.
- DEVICE-PENDING: the code itself flags that the iOS real-ICMP backend "cannot be verified without a real device". The logic and gating are unit-tested with a fake backend; the live round-trip is the device-pending piece (icmp_service.dart:24-27, 47-52). On desktop, use TCP Ping.

Source: `icmp_service.dart:1-303`

Ping (TCP)

A reachability + round-trip-latency probe that works on every platform, including the sandboxed desktop, by timing a TCP handshake, not ICMP echo.

WHY IT'S HERE

The portable ping. ICMP is often filtered while a TCP port (443) answers; this is the tcping/paping approach pros already use. It is also the desktop path where real ICMP can't run.

HOW TO USE

1. Enter a host, optionally pick a probe port (443 default; presets 443/80/53/22/7) and count, run. Live min/avg/max/loss and a sparkline build as replies land.

FORMULA OR METHOD

> Each "ping" is a timed Socket.connect to host:port (default 443). A completed handshake OR an actively-refused RST both count as a successful round trip for latency (exactly how tcping treats it); only a genuine timeout or lookup failure is a loss (ping_service.dart:1-27, 158-161, 214-247). Probes are spaced by the requested interval minus the time the probe took, so cadence stays steady under latency (ping_service.dart:194-204).

FIELD NOTES

- Platform differences: identical everywhere native. Gated off on web. The screen labels the metric "TCP RTT" and shows the target port so it's never mistaken for ICMP. (ping_service.dart:19-22)
- RTT is the TCP handshake time to a port, slightly higher than ICMP and dependent on the chosen port answering. A "refused" target still gives you a valid latency number (the host answered).
- This is not ICMP echo (see Ping (ICMP) for the mobile real-ICMP path). If a host filters the probe port, it reads as loss even if the host is up on another port; try a different probe port.

Source: `ping_service.dart:1-247`

Ping Plotter

Runs a sustained ping to a target and charts round-trip latency over time, instead of the single-shot result the Ping and ICMP Ping tools give. The live trend, jitter, and visible dropped probes show how stable a path is, not just whether it answers once.

WHY IT'S HERE

The live performance graph. A single ping says "reachable now"; a trend says "steady, spiky, or dropping packets." Reach for this to watch a flaky link over seconds or minutes, the view the single-shot ping tools can't give.

HOW TO USE

1. Enter a host, optionally pick a probe port (443 default; presets 443/80/53/22/7) and a sample interval (0.5s / 1s / 2s / 5s), then Start plot.
2. The chart fills left-to-right as replies land: a lime line for RTT and a red dot on the axis for any lost probe. The readout above shows current / min / avg / max / jitter and loss%.
3. It runs until you tap Stop; the chart keeps the most recent samples (a bounded window) so a long run stays fixed-size. Copy exports the summary plus a per-sample table.

FORMULA OR METHOD

> Drives the shipped TCP-handshake Ping engine (PingService) in continuous mode (count = 0), one probe per chosen interval. Each reply folds into a bounded rolling window (default last 60 samples); min/avg/max are over the landed RTTs in that window, jitter is the mean absolute difference between consecutive landed RTTs (a lost probe breaks the chain, so jitter never pairs across a gap), and loss% is lost/sent. A timed-out / unreachable probe is recorded as an honest gap (no RTT) and drawn as a red axis dot, never as a fabricated 0 ms (ping_plot_controller.dart; ping_service.dart:1-27, 169-212).

FIELD NOTES

- This is a TCP round-trip probe, not ICMP echo. The metric is labeled "TCP RTT" and the probe port is shown, so it is never mistaken for ICMP (see Ping (ICMP) for the mobile real-ICMP path).
- Platform: runs anywhere native, including the sandboxed macOS desktop where real ICMP can't (the ICMP path needs a subprocess the App Sandbox blocks). Gated off on web with the download-the-app prompt.
- Dropped probes are shown, never hidden: a lost sample is a red dot on the axis and counts toward loss%, so a flaky path reads honestly instead of as a smooth line.
- The chart retains a bounded window of recent samples (so memory stays flat on a long run); the copy export notes how many of the total samples are shown.

Source: `ping_plotter_screen.dart; ping_plot_controller.dart; ping_service.dart:1-247`

Ping Sweep

Discover responsive hosts on a subnet via a TCP-probe sweep (no ICMP).

WHY IT'S HERE

A quick "who's on this segment" without raw sockets or a subprocess. Reach for it to enumerate live hosts on a /24.

HOW TO USE

1. Enter a CIDR (192.168.1.0/24), a range (192.168.1.10-40 or full end address), or a single IP; run. A live progress bar and a running responsive count build as hosts settle.

FORMULA OR METHOD

> For each candidate, a TCP Socket.connect to a common port (443 default; the sweep can probe 443/80/22/53 and a host is responsive the moment ANY answers — first-answer wins). A completed handshake or a refused RST both prove the host answered; a timeout means silent on that port (ping_sweep_service.dart:158-165, 380-406). Bounded worker pool (default 32 in flight) (ping_sweep_service.dart:28-31, 309-375). Hard cap of 254 hosts (a /24); anything larger is rejected with "that's N hosts, the cap is M" — never silently truncated (ping_sweep_service.dart:166-171, 222-229).

FIELD NOTES

- Platform differences: identical on every native platform. Gated off on web. (ping_sweep_service.dart:33-34)
- A host is reported "responded", NOT "up"; a host silent on the probed ports may still be alive (ICMP-only or firewalled). The tool never claims ICMP-style liveness. (ping_sweep_service.dart:21-26, 94-100)
- This finds hosts that answer TCP on the probed ports. For richer host detail (name, services, type, vendor), use Network Discovery. CIDR /31 and /32 include every address; larger blocks exclude network and broadcast.

Source: `ping_sweep_service.dart:21-406`

Port Scan

TCP connect scan of a host, either a common-ports preset or a custom range, reporting each port open/closed/filtered.

WHY IT'S HERE

A privilege-free nmap -sT for the field. Reach for it to see what services a host exposes.

HOW TO USE

1. Enter a host, pick the common-ports preset or type a custom spec (e.g. 22, 80, 443, 8000-8100), run. Results stream in as ports settle.

FORMULA OR METHOD

```
Per port, Socket.connect(host, port) with an 800ms default timeout. Open = handshake completes; Closed = actively refused/reset (host reachable, nothing listening); Filtered = no response before timeout (a firewall dropping the SYN). Same open/closed/filtered taxonomy nmap reports for a connect scan, with no raw socket (port_scan_service.dart:1-33, 272-310). Connects run in a bounded worker pool (default 64 in flight) and stream incrementally (port_scan_service.dart:200-270). The common-ports preset is 44 curated ports a network pro actually checks (20-27017, with service labels like 443→HTTPS, 3389→RDP) (port_scan_service.dart:101-160).
```

FIELD NOTES

- Platform differences: works identically on every native platform (no entitlement beyond network-client). Gated off on web. (port_scan_service.dart:17-18)
- "Filtered" means the SYN went unanswered (likely a firewall); it does not mean the port is closed. A custom range parser de-dupes and bounds-checks (1 to 65535). (port_scan_service.dart:165-188)
- This is a TCP connect scan, not a SYN/stealth scan; it completes the handshake then tears it down. A host that's all-filtered is usually unreachable or firewalled wholesale.

Source: `port_scan_service.dart:1-310`

Traceroute (Mobile)

Hop-by-hop path via an ICMP TTL-walk, Android only (iOS unsupported).

WHY IT'S HERE

Extends traceroute to mobile where the platform genuinely supports it, built on the same shared ICMP layer as Ping (ICMP).

HOW TO USE

1. Android: enter a host, run; hops fill in via a TTL-walk (one ICMP echo per increasing TTL, surfacing the router that answers each). (icmp_service.dart:387-450)

FORMULA OR METHOD

```
A TTL-walk on the ICMP layer: send echoes with TTL 1..maxHops; an intermediate router answering TimeExceeded names that hop; the target answering EchoReply ends the walk (icmp_service.dart:73-93, 398-472).
```

FIELD NOTES

- Platform matrix (the critical honesty point): Android, available; dart_ping's TTL maps to ping -t <ttl> (outbound TTL) and the system ping prints the responding hop on a "Time to live exceeded" line (icmp_service.dart:29-35, 308-313). iOS, not feasible, honestly unavailable. iOS can echo (via GBPing), but GBPing's receive path only accepts ICMP EchoReply (type 0); it never parses TimeExceeded (type 11), the message a traceroute needs to name each hop. Setting a low TTL just makes the echo time out with no hop IP. So iOS gets an honest "not on this device", never faked hops (icmp_service.dart:16-26, 84-86, 308-313). Desktop: the system traceroute is the path; this ICMP TTL-walk reports sandboxed-desktop (icmp_service.dart:87-92).
- Where available (Android), hops are the same TTL/IP/RTT/* * * shape as the system traceroute.
- The iOS limitation is a real platform ceiling in GBPing, documented at length in the source; it is the reason this tool is Android-only. DEVICE-PENDING: the Android path is device-pending verification per the source (icmp_service.dart:29-35).

Source: `icmp_service.dart:16-472`

Traceroute (System)

Hop-by-hop path discovery via the OS traceroute/tracert (desktop).

WHY IT'S HERE

The genuine traceroute, where it can actually run. Reach for it on a Mac/PC to see the routed path and where latency or loss enters.

HOW TO USE

1. Desktop only. Enter a host, run; hops fill in live as the OS tool emits them. Cancellable mid-flight.

FORMULA OR METHOD

> `Spawns the system traceroute (Unix: -m maxHops -q 3 -w 2) or tracert (Windows: -d -h maxHops -w 2000) and parses each hop line live from stdout/stderr — TTL, host/IP, per-probe RTTs, and * * * timeouts (traceroute_service.dart:194-302, 304-386). A real traceroute needs to read ICMP TIME_EXCEEDED replies, which require either a raw socket or the privileged system binary — so faking hops from TCP timing is explicitly refused (traceroute_service.dart:1-31).`

FIELD NOTES

- Platform matrix (honest): macOS / Windows / Linux desktop spawns the OS binary. But under the macOS App Sandbox (the App Store build) spawning is blocked, so the screen runs a live isLaunchable() probe (a side-effect-free no-arg launch) and adapts: a non-sandboxed Developer-ID macOS build and Windows/Linux launch it fine; the sandboxed build shows an explicit "binary unavailable" verdict rather than hanging or pretending (traceroute_service.dart:151-187, 228-246). iOS / Android: subprocess execution is sandboxed out entirely → "Traceroute runs on desktop, use Ping here." (traceroute_service.dart:202-215). Web: never reached (gated).
- Each hop shows TTL, the responding router (name + IP), and up to three probe RTTs; * * * is a hop that didn't answer (common and not necessarily a problem). Reaching the target is reported as a terminal "complete".
- On a sandboxed macOS App Store build this tool honestly reports unavailable. For a path read on mobile, see Traceroute (Mobile), but note its iOS limitation.

Source: `traceroute_service.dart:1-386`

Wake-on-LAN

Send a Wake-on-LAN magic packet to wake a host by MAC address.

WHY IT'S HERE

Wake a sleeping machine on the LAN from your phone or laptop.

HOW TO USE

1. Enter the target MAC (colons, hyphens, Cisco dots, or no separators all parse), optionally a subnet-directed broadcast (e.g. 192.168.1.255) and port (9 default, 7 alternative), send.

FORMULA OR METHOD

> `Builds the 102-byte magic packet (6× 0xFF then the 6-byte MAC repeated 16 times) and sends it as a UDP broadcast via RawDatagramSocket with broadcastEnabled = true — no subprocess, no privileged socket (wake_on_lan_service.dart:1-21, 92-108, 218-233). The MAC is normalized to canonical form; an invalid MAC, bad broadcast IP, or out-of-range port is rejected with a clear message (wake_on_lan_service.dart:118-205).`

FIELD NOTES

- Platform differences: works on every native platform (outbound UDP broadcast is covered by the existing entitlements). Gated off on web. (wake_on_lan_service.dart:20-21)
- Success means the packet was sent; it makes NO claim the device woke. WoL is unacknowledged, a switch may not forward the all-ones broadcast across subnets, and the target may have WoL disabled (wake_on_lan_service.dart:13-18, 26-27). The tool shows the bytes sent and the packet hex.
- If the OS reports 0 bytes sent, it suggests a directed broadcast (e.g. 192.168.1.255) instead of 255.255.255.255. "Sent" ≠ "woke"; verify the host separately.

Source: `wake_on_lan_service.dart:1-233`

WHOIS

Domain/IP registration lookup over WHOIS (TCP port 43), with parsed highlights and the raw record.

WHY IT'S HERE

Registrar, dates, status, and name servers for a domain or IP, from the field. Reach for it on ownership/expiry questions.

HOW TO USE

1. Enter a domain or IP, look up.

FORMULA OR METHOD

> Raw WHOIS over TCP/43 via Socket.connect — not a whois subprocess (sandbox-blocked) and not RDAP/HTTPS (uneven coverage, no CORS) (whois_service.dart:1-32). It does the hierarchical two-hop dance: query whois.iana.org for the target, parse the refer:/whois: referral to the authoritative registry, re-query that, and follow one optional further hop to a Registrar WHOIS Server: if it returns a fuller record (whois_service.dart:21-32, 178-258). Highlights (registrar, created/updated/expires, status, name servers) are parsed from the free-form record where reliably present; anything missing is omitted, never faked (whois_service.dart:300-353).

FIELD NOTES

- Platform differences: same on every native platform. Gated off on web. (whois_service.dart:31-32)
- Three states: success (raw record + highlights), empty (server answered but the object is unregistered / a "No match" banner), and failure (connection/timeout/bad input) (whois_service.dart:52-132). The servers consulted are listed so the path is transparent.
- WHOIS output is registry-specific and free-form; highlights are best-effort. The raw record is the source of truth.

Source: whois_service.dart:1-353

CATEGORY

Calculators & Tools

26 tools

RF math and field utilities. Link-budget building blocks, antenna and coverage geometry, capacity and power figures, coordinate work, and unit conversions, each computed locally on the device.

RF & Propagation 9

Cable Loss

Estimates total coax attenuation for a run of a known cable type, length, and frequency, plus the per-100ft loss coefficient at that frequency.

WHY IT'S HERE

Cable loss is a direct subtraction in the transmit and receive chains. Pick a cable type and length and the tool tells you how much signal the feedline will eat.

HOW TO USE

1. Pick the cable type from the list.
2. Enter frequency and pick its unit (GHz or MHz).
3. Enter run length and pick its unit (ft or m).
4. Read total loss in dB plus the per-100ft coefficient.

INPUTS

INPUT	UNIT	RANGE
Cable type	one of: LMR-100A, LMR-200, LMR-400 (default), LMR-600, LMR-900, LMR-1200, RG-58, RG-8/U, RG-213, RG-214 (:60-74)	—
Frequency	GHz (default) or MHz	must be > 0
Length	ft (default) or m	must be > 0

FORMULA OR METHOD

> Loss per 100 ft is interpolated across manufacturer [freq_MHz, dB/100ft] knot points on a sqrt(frequency) axis: t = (sqrt(f) − sqrt(f1)) / (sqrt(f2) − sqrt(f1)), loss = l1 + t·(l2 − l1) (:200-210). Below the lowest knot it clamps to the lowest value; above the top knot it sqrt-extrapolates from the last two knots (:188-198). total_loss(dB) = per100ft · length_ft / 100 (:217-219). GHz × 1000 → MHz; m × 3.28084 → ft (:160-178). Output rounded to 2 decimals. Per-cable knot tables are ported verbatim (:77-155); e.g. LMR-400 at 2400 MHz = 3.9 dB/100ft. (cable_loss_screen.dart:160-219)

EXAMPLE

> LMR-400, 2.4 GHz, 50 ft → per-100ft at 2400 MHz = 3.9 dB; total = 3.9 · 50 / 100 = 1.95 dB.

FIELD NOTES

- Values are manufacturer-typical at room temperature; real loss rises with temperature and aging, and connectors add loss not modeled here.
- The sqrt(f) interpolation is a smooth fit between published spec points, not a measurement.
- Extrapolation above the highest published frequency (e.g. above 5800 MHz for LMR cables) is a model estimate. Treat 6 GHz results with caution.

Source: `cable_loss_screen.dart:160-219`

Earth Curvature

Computes the earth bulge (the height of the earth's curvature at the midpoint of a path) for a given path length and atmospheric K-factor.

WHY IT'S HERE

On long microwave links the earth's curvature, modified by atmospheric refraction, eats into Fresnel clearance. This sizes how much extra antenna height the curvature demands.

HOW TO USE

1. Enter path length and pick its unit (km or mi).
2. Pick the K-factor.
3. Read the bulge in meters and feet.

INPUTS

INPUT	UNIT	RANGE
Path length	`km (default) or mi`	`mi × 1.60934 → km`
K-factor	`select`	`4/3 standard (1.333, default), 1.0 geometric, 2/3 worst-case (0.667), 2.0 superrefraction (:42-45)`

FORMULA OR METHOD

> `bulge(m) = d_km² · 1000 / (8 · 6371 · K) (mean earth radius 6371 km, scaled by K-factor) (:73-79). bulge(ft) = bulge(m) · 3.28084 (:82-83). (earth_curvature_screen.dart:73-83)`

EXAMPLE

> 20 km path, K = 1.333 → 20² · 1000 / (8 · 6371 · 1.333) = 5.89 m.

FIELD NOTES

- K = 4/3 is the standard-atmosphere assumption; K = 2/3 is the conservative worst case used for availability-critical links (smaller K means more bulge).
- The bulge is the maximum at midpoint; add it to required Fresnel clearance when sizing tower heights.
- Atmospheric K-factor varies with weather and geography. This is a planning value, not a guarantee.

Source: `earth_curvature_screen.dart:73-83`

Free Space Path Loss

Computes the loss in dB a signal suffers traveling through free space between transmitter and receiver, given the operating frequency and distance.

WHY IT'S HERE

The starting point of any link budget. Reach for it to estimate how much signal a point-to-point or outdoor link loses before you factor in antennas and cables.

HOW TO USE

1. Enter the frequency and pick its unit (GHz or MHz).
2. Enter the distance and pick its unit (km, mi, or m).
3. Read the path loss in dB. The result blanks to "—" until both fields hold valid positive numbers.

INPUTS

INPUT	UNIT	RANGE
Frequency	`GHz (default) or MHz`	`must be > 0`
Distance	`km (default), mi, or m`	`must be > 0`

FORMULA OR METHOD

> `FSPL(dB) = 20·log10(f_GHz) + 20·log10(d_km) + 92.45 (fspl_screen.dart:71-75). Unit conversions: MHz ÷ 1000 → GHz (:48-55); mi × 1.60934 → km, m ÷ 1000 → km (:58-67). The 92.45 constant is the km/GHz form of FSPL. Output is rounded to 1 decimal (:138-141). If frequency or distance ≤ 0, output blanks (:122-125).`

EXAMPLE

> 5 GHz, 1 km → 20·log10(5) + 20·log10(1) + 92.45 = 106.4 dB.

FIELD NOTES

- Free space only. No obstructions, no ground reflection, no atmospheric effects. Real-world links always lose more, so use this as a floor, not a prediction.
- The reference card lists anchor values (2.4 GHz @ 1 km = 100.1 dB, 6 GHz @ 1 km = 108.0 dB).

Source: `fspl_screen.dart:71-75`

Fresnel Zone

Computes the first Fresnel zone radius along a point-to-point path, plus the 60% clearance value planners treat as the "keep it clear" threshold.

WHY IT'S HERE

Line of sight is not enough for a reliable link; the first Fresnel zone must be mostly clear of obstructions. This sizes how much clearance you need above terrain, rooftops, and trees.

HOW TO USE

1. Enter frequency in GHz.
2. Enter total path distance in meters.
3. Optionally enter a point-from-TX distance in meters to get the radius at that specific point; leave blank for the midpoint (maximum) only.
4. Read first-zone radius and 60% clearance (in meters, feet shown alongside).

INPUTS

INPUT	UNIT	RANGE
Frequency	GHz (fixed unit)	must be > 0
Total path distance	meters	must be > 0
Point from TX	meters, optional	must be inside (0, total) to add an at-point result, otherwise ignored

FORMULA OR METHOD

λ(m) = 0.3 / f_GHz (c/f form, c = 3·10⁸). At-point radius: r = sqrt(λ · d1 · d2 / (d1 + d2)) where d1, d2 are the two path segments in meters. Midpoint (the maximum, always shown): d1 = d2 = D/2. clearance60 = r · 0.6. Feet = r · 3.28084. (fresnel_screen.dart:73-117)

EXAMPLE

5.8 GHz, 10000 m total, midpoint → λ = 0.3/5.8 = 0.05172 m; r = sqrt(0.05172 · 5000 · 5000 / 10000) = 11.37 m; 60% clearance = 6.82 m.

FIELD NOTES

- The midpoint radius is the worst case along the path; clearance there is the binding constraint.
- Frequency unit is fixed to GHz here (unlike FSPL, which offers MHz).
- 60% clearance is the common rule of thumb; some designs require more in heavy-foliage or reflective environments.

Source: fresnel_screen.dart:73-117

ITU Rain Fade

Estimates rain attenuation on a microwave link using ITU-R P.838-3 (specific attenuation) and a simplified ITU-R P.530 (effective path length).

WHY IT'S HERE

Above ~10 GHz, rain is the dominant fade mechanism on outdoor links. This sizes the fade margin a backhaul link needs to survive heavy rain.

HOW TO USE

1. Enter frequency in GHz.
2. Enter rain rate in mm/hr.
3. Enter path length and pick its unit (km or mi).
4. Pick polarization (Horizontal or Vertical).
5. Read total rain attenuation, specific attenuation γ, and effective path length.

INPUTS

INPUT	UNIT	RANGE
Frequency	GHz (fixed)	must be > 0
Rain rate	mm/hr	must be > 0
Path length	km (default) or mi	must be > 0
Polarization	Horizontal or Vertical	—

FORMULA OR METHOD

> k and α come from the ITU-R P.838-3 [freq, kH, αH, kV, αV] table (18 frequency nodes, 1-100 GHz, :62-81), interpolated log-log on frequency for k and log-linear on α; clamped at the table ends (:100-131). Specific attenuation: γ(dB/km) = k · R^α (:134-141). Effective path length: L_eff = L / (1 + L/d0) with d0 = 35 · e^(−0.015·R) (:145-148). Total: attenuation(dB) = γ · L_eff (:151-160). Output decimals: attenuation 2, γ 4, L_eff 2. (rain_fade_screen.dart:62-160)

EXAMPLE

> 23 GHz, 42 mm/hr, 5 km, Horizontal → interpolating between the 20 and 25 GHz nodes gives k ≈ 0.1027, α ≈ 1.085; γ = 0.1027 · 42^1.085 = 5.7195 dB/km; d0 = 35·e^(−0.63) = 18.65, L_eff = 5/(1 + 5/18.65) = 3.94 km; attenuation = 5.7195 · 3.94 = 22.55 dB.

FIELD NOTES

- The model assumes the rain rate is uniform across the path; real cells are smaller, which the L_eff reduction partly accounts for.
- Pick a rain rate matching your target availability (e.g. the 0.01%-of-time rate for your region). The tool does not supply regional rain statistics.
- Below ~10 GHz rain fade is usually negligible.

Source: `rain_fade_screen.dart:62-160`

Link Budget

Full point-to-point link budget: combines transmit power, antenna gains, cable losses, path loss, and miscellaneous losses into a received signal level, then compares against receiver sensitivity for a fade margin.

WHY IT'S HERE

The decision tool for whether a link will close, and by how much. Assemble all the gains and losses to see the margin before you deploy.

HOW TO USE

1. Enter TX power and pick its unit (dBm, W, or mW).
2. Enter TX antenna gain, TX cable loss, path loss, RX cable loss, RX antenna gain, and RX sensitivity.
3. Optionally enter other/miscellaneous losses (treated as 0 when blank).
4. Read received signal (dBm) and link margin (dB), color-coded by health.

INPUTS

INPUT	UNIT	RANGE
TX power	dBm (default), W, or mW	W/mW must be > 0
TX gain	dBi	—
TX loss	dB	—
Path loss	dB	—
RX loss	dB	—
RX gain	dBi	—
RX sensitivity	dBm	—
Other losses	dB	optional, default 0

FORMULA OR METHOD

> Watts → dBm: 10·log10(W·1000); mW via W = mW/1000 (:56-71). received(dBm) = TX_power + TX_gain − TX_loss − path_loss − RX_loss + RX_gain − misc (:76-86). link_margin(dB) = received − RX_sensitivity (:89-91). Health bands (:94-98): margin ≥ 10 dB → healthy (green); 0 to 10 dB → marginal (amber); < 0 dB → negative (red). (link_budget_screen.dart:56-98)

EXAMPLE

> TX 30 dBm, TX gain 20, TX loss 2, path loss 120, RX loss 2, RX gain 20, misc 0, RX sensitivity −85 → received = 30 + 20 − 2 − 120 − 2 + 20 − 0 = −54 dBm; margin = −54 − (−85) = 31 dB (healthy).

FIELD NOTES

- Path loss is supplied by the engineer (compute it with the FSPL tool, then add real-world margins).
- This budget does not include rain fade; use the PtP Link Check for that.
- A healthy margin (≥ 10 dB) is the common design target to ride out fading and multipath.

Source: `link_budget_screen.dart:56-98`

Noise Floor

Computes the thermal noise floor (kTB) for a channel, the receiver noise floor (kTB plus noise figure), and the quick −174 dBm/Hz rule-of-thumb value.

WHY IT'S HERE

SNR is signal minus noise floor. Knowing the noise floor for a given channel width sets the minimum usable signal and frames link-margin discussions.

HOW TO USE

1. Pick the channel bandwidth.
2. Enter receiver noise figure in dB.
3. Optionally adjust temperature (defaults to 20°C when blank).
4. Read thermal noise floor, receiver noise floor, and the rule-of-thumb value.

INPUTS

INPUT	UNIT	RANGE
Bandwidth	MHz	select of 20, 40, 80, 160, or 320 MHz (:44-49)
Noise figure	dB	must be ≥ 0 (default 7; invalid/negative blanks the outputs)
Temperature	°C	defaults to 20 when blank (:69-70)

FORMULA OR METHOD

> thermal(dBm) = 10·log10(k · T · bw_Hz) + 30, with k = 1.380649·10⁻²³ J/K, T = tempC + 273.15, bw_Hz = bw_MHz · 10⁶ (:74-78). rx_floor(dBm) = thermal + noise_figure (:81-83). rule(dBm) = −174 + 10·log10(bw_Hz) (the −174 dBm/Hz constant is kTB at ~0°C) (:87-90). Output rounded to 1 decimal. (noise_floor_screen.dart:66-90)

EXAMPLE

> 20 MHz, NF 7 dB, 20°C → thermal = 10·log10(1.380649e-23 · 293.15 · 20e6) + 30 = −100.9 dBm; rx floor = −93.9 dBm; rule = −174 + 10·log10(20e6) = −101.0 dBm.

FIELD NOTES

- This is the theoretical noise floor; real receivers see a higher effective floor from co-channel interference, adjacent-channel leakage, and ambient RF.
- The −174 rule is a 0°C approximation and differs slightly from the temperature-aware thermal value (note the HTML input prefills 25°C but the code's blank-field fallback is 20°C, per :69-70).

Source: noise_floor_screen.dart:66-90

PtP Link Check

Full point-to-point backhaul link budget end to end, from TX power through antenna gains, cable losses, free-space path loss, and rain fade to the received signal, compared against sensitivity for a fade margin and a PASS / MARGINAL / FAIL verdict.

WHY IT'S HERE

The all-in-one decision tool for a microwave or Wi-Fi backhaul link, combining FSPL and rain fade with the link budget so you get a single go/no-go answer.

HOW TO USE

1. Enter frequency (GHz), distance (km/mi), TX power (dBm), TX gain (dBi), RX gain (dBi), and RX sensitivity (dBm).
2. Optionally enter TX loss, RX loss, rain rate, required margin, and pick polarization.
3. Read EIRP, free-space loss, rain fade, received signal, link margin, and the verdict.

INPUTS

INPUT	UNIT	RANGE
Frequency	GHz (fixed)	must be > 0
Distance	km (default) or mi	must be > 0
TX power	dBm	signed
RX sensitivity	dBm	signed; sensitivity usually negative
TX gain	dBi	—
RX gain	dBi	—
TX loss	dB	default 0
RX loss	dB	default 0
Rain rate	mm/hr	default 0 (no fade)
Required margin	dB	default 10
Polarization	Horizontal or Vertical	—

FORMULA OR METHOD

eirp = TX_power + TX_gain − TX_loss (:223). fspl = 20·log10(d_km) + 20·log10(f_GHz) + 92.45 (:155-157). Rain fade reuses the ITU-R P.838-3 table and the same log-log interpolation as the Rain Fade tool: γ = k·R^α, d0 = 35·e^(−0.015·R), L_eff = d/(1 + d/d0), rainFade = γ·L_eff; returns 0 when rain rate is not > 0 (:191-205). rxLevel = eirp − fspl − rainFade + RX_gain − RX_loss (:226). margin = rxLevel − RX_sensitivity (:227). Binary pass: margin ≥ required_margin (:234). Three-state verdict (:239-243): margin ≥ required → PASS; 0 ≤ margin < required → MARGINAL; margin < 0 → FAIL. Output decimals: EIRP 1, FSPL 1, rain fade 2, RX level 1, margin 1. (ptp_link_screen.dart:154-243)

EXAMPLE

5.8 GHz, 10 km, TX 20 dBm, TX gain 23, RX gain 23, sensitivity −78, TX/RX losses 0, rain 0, required margin 10 → EIRP = 20 + 23 − 0 = 43.0 dBm; FSPL = 20·log10(10) + 20·log10(5.8) + 92.45 = 127.7 dB; rxLevel = 43 − 127.7 − 0 + 23 − 0 = −61.7 dBm; margin = −61.7 − (−78) = 16.3 dB → PASS. (With TX loss 1 and RX loss 1: EIRP 42.0, rxLevel −63.7, margin 14.3, still PASS.)

FIELD NOTES

- This is the most complete link tool in the suite. Prefer it over the bare Link Budget when the link is outdoors and above ~10 GHz.
- FSPL alone underestimates real loss; the rain fade leg only covers rain, not fog, foliage, or multipath.
- The MARGINAL band (link closes but below your required margin) is the tool's own warning state; the underlying PASS/FAIL is purely margin ≥ required.

Source: ptp_link_screen.dart:154-243

RF Attenuation

Estimates total path loss through building materials by summing per-layer attenuation for a chosen Wi-Fi band.

WHY IT'S HERE

Indoor coverage planning. Count the walls, floors, and obstructions between AP and client and get a quick total dB loss to add to the link budget.

HOW TO USE

1. Pick the band (2.4, 5, or 6 GHz).
2. Enter a quantity for each material the signal passes through.
3. Read the total attenuation in dB.

INPUTS

INPUT	UNIT	RANGE
Band	`select`	`2.4 GHz, 5 GHz, or 6 GHz`
Per-material quantities	`integers`	`only quantities > 0 contribute`

FORMULA OR METHOD

> `Each material has a fixed per-layer loss per band from a ported table (e.g. Drywall 3/4/5 dB at 2.4/5/6 GHz; Concrete poured 13/16/19; Metal door 20/26/30; Foil insulation 25/30/35) (materials const, dataset section). total(dB) = Σ (loss_per_layer[band] · qty) over all materials with qty > 0. (rf_attenuation_screen.dart, dataset + math)`

EXAMPLE

> 5 GHz, 2 sheets of drywall (4 dB each) + 1 concrete block (13 dB) → 2·4 + 1·13 = 21 dB.

FIELD NOTES

- The per-material values are typical attenuation figures, not measurements of your building. Real walls vary widely with construction, moisture, and reinforcement.
- Metal, foil/vapor barriers, and low-E glass are near-total blockers and dominate the total.
- Use this for a first-pass coverage estimate, then validate with a survey.

Source: `rf_attenuation_screen.dart`

Antenna & Coverage 4

Antenna Downtilt

Computes the mechanical downtilt angle that aims an antenna's beam center at a target coverage distance on the ground.

WHY IT'S HERE

When mounting a sector or directional antenna at height, you need the tilt angle to put the main lobe where the users are.

HOW TO USE

1. Enter antenna height above ground (AGL) and pick its unit (ft or m).
2. Enter target coverage distance and pick its unit (km, ft, or m).
3. Read the downtilt angle in degrees.

INPUTS

INPUT	UNIT	RANGE
Antenna height (AGL)	`ft or m`	—
Target coverage distance	`km, ft, or m`	—

FORMULA OR METHOD

> `downtilt(deg) = atan(height_m / coverage_m) · 180/π. Unit conversions: ft × 0.3048 → m; km × 1000 → m. (downtilt_screen.dart, math section)`

EXAMPLE

> 30 m height, 200 m coverage → atan(30/200) · 180/π = 8.53°.

FIELD NOTES

- This aims the beam center at the target distance; it does not account for the antenna's vertical beamwidth (use Downtilt Coverage for the near/far edges).
- Pure geometry, no consideration of antenna pattern shape or ground slope.

Source: `downtilt_screen.dart`

Downtilt Coverage

Computes the near and far ground coverage edges (and coverage depth) of a downtilted antenna, given height, tilt angle, and vertical beamwidth.

WHY IT'S HERE

The complement to the Downtilt tool. It shows the footprint the beam actually covers, so you can check for coverage gaps or overshoot.

HOW TO USE

1. Enter antenna height (AGL) and pick its unit (ft or m).
2. Enter the downtilt angle in degrees.
3. Enter the antenna's vertical beamwidth in degrees.
4. Read near edge, far edge, and coverage depth.

INPUTS

INPUT	UNIT	RANGE
Antenna height (AGL)	ft or m	must be > 0
Downtilt angle	degrees	—
Vertical beamwidth	degrees	must be in the open range (0, 180)

FORMULA OR METHOD

> farAngle = tilt − beamwidth/2, nearAngle = tilt + beamwidth/2 (radians). nearEdge = height / tan(nearAngle). If farAngle ≤ 0 the upper beam edge is at or above the horizon → far edge is unbounded (reported as beam-above-horizon, no far edge or depth). Otherwise farEdge = height / tan(farAngle), depth = farEdge − nearEdge. Returns null (blanks) if height ≤ 0 or beamwidth outside (0, 180). (downtilt_coverage_screen.dart, math section)

EXAMPLE

> 30 m height, 10° tilt, 8° beamwidth → farAngle = 6°, nearAngle = 14°; nearEdge = 30/tan(14°) = 120.3 m; farEdge = 30/tan(6°) = 285.4 m; depth = 165.1 m.

FIELD NOTES

- When the tilt is shallow relative to beamwidth (e.g. tilt 6°, beamwidth 15° → farAngle = −1.5°), the upper edge clears the horizon and the far edge is unbounded. The tool flags this rather than returning a negative distance.
- Uses beam edges at the half-power (or stated) beamwidth; actual coverage tails off gradually beyond those edges.

Source: downtilt_coverage_screen.dart

EIRP Calculator

Computes Effective Isotropic Radiated Power, the actual power radiated from an antenna system after accounting for transmitter power, cable/connector loss, and antenna gain.

WHY IT'S HERE

EIRP is the regulatory ceiling number. Check it against FCC/ETSI limits and use it as the transmit-side input to a link budget.

HOW TO USE

1. Enter TX power and pick its unit (dBm, W, or mW).
2. Enter cable loss in dB and antenna gain in dBi.
3. Read EIRP in dBm, plus a secondary line in watts/milliwatts.

INPUTS

INPUT	UNIT	RANGE
TX power	dBm (default), W, or mW	W/mW must be > 0 (a non-positive value makes the log non-finite and blanks the result)
Cable loss	dB	signed allowed
Antenna gain	dBi	signed allowed

FORMULA OR METHOD

> Power → dBm: dBm = 10·log10(W·1000) for watts; mW routes through the same with W = mW/1000 (:42-58). EIRP(dBm) = TX_power_dBm − cable_loss + antenna_gain (:63-72). EIRP(W) = 10^(EIRP_dBm/10) / 1000 (:42-43, 75). EIRP output 1 decimal; the watt line shows W at 2 decimals when ≥ 1 W, else mW at 1 decimal (:149-160). (eirp_screen.dart:42-75)

EXAMPLE

> TX 20 dBm, cable loss 1.5 dB, gain 14 dBi → 20 − 1.5 + 14 = 32.5 dBm → 10^(3.25)/1000 = 1.78 W.

FIELD NOTES

- Cable loss should include all connector and jumper losses on the transmit chain.
- The reference card lists regulatory EIRP ceilings (e.g. 2.4 GHz FCC PtMP = +36 dBm / 4 W); those are planning anchors and vary by sub-band, channel width, and power-control rules. Verify against current local regulations.

Source: eirp_screen.dart:42-75

Wavelength

Converts a frequency to its wavelength in meters, centimeters, feet, and inches.

WHY IT'S HERE

Antenna element sizing, ground-plane and spacing rules, and quarter-wave / half-wave rules of thumb all key off wavelength. A quick reference when laying out antennas.

HOW TO USE

1. Enter the frequency and pick its unit (MHz or GHz).
2. Read the wavelength in all four units at once.

INPUTS

INPUT	UNIT	RANGE
Frequency	`MHz (default) or GHz`	`must be > 0`

FORMULA OR METHOD

```
λ(m) = 300 / f_MHz (the 300 constant is c = 3·10⁸ m/s scaled for the MHz/meter form).
λ(cm) = λ(m)·100, λ(ft) = λ(m)·3.28084, λ(in) = λ(ft)·12. GHz × 1000 → MHz (:45-52).
Display decimals: m → 4, cm → 2, ft → 4, in → 3 (:209-212). (wavelength_screen.dart:54-67)
```

EXAMPLE

2400 MHz → 300/2400 = 0.1250 m, 12.50 cm, 0.4101 ft, 4.921 in.

FIELD NOTES

- Uses c = 3·10⁸ m/s (the rounded 300 constant), not the exact 299792458. The tiny difference is irrelevant for antenna work.
- This is free-space wavelength; velocity factor inside cable or dielectric is not applied.

Source: `wavelength_screen.dart:54-67`

Capacity & Power 3

Capacity Planner

Recommends the number of access points needed for a space, based on user count, concurrency, per-user demand, AP capacity, target utilization, and an optional clients-per-AP density cap.

WHY IT'S HERE

High-density design. Size AP count by both throughput demand and client-density limits, taking the larger of the two.

HOW TO USE

1. Enter total users, concurrent usage %, per-user throughput (Mbps), AP max throughput (Mbps), and target channel utilization %.
2. Optionally enter max clients per AP for a density check.
3. Read concurrent users, total bandwidth demand, APs by throughput, APs by density, and the recommended AP count.

INPUTS

INPUT	UNIT	RANGE
Total users	count	must be > 0 (default hint 200)
Concurrent usage	%	must be > 0 (default hint 70)
Per-user throughput	Mbps	must be > 0 (default hint 5)
AP max throughput	Mbps	must be > 0 (default hint 600)
Target channel utilization	%	must be > 0 (default hint 50)
Max clients per AP	count	optional (default hint 50); ≤ 0 or blank disables the density check

FORMULA OR METHOD

> concurrent = ceil(users · concurrentPct/100). totalBw = concurrent · perUserMbps. effectiveAp = apMaxMbps · targetUtilPct/100. apsByThroughput = ceil(totalBw / effectiveAp). apsByDensity = ceil(concurrent / maxClients) (0 when no cap). recommended = max(apsByThroughput, apsByDensity, 1). (capacity_planner_screen.dart, compute)

EXAMPLE

> 200 users, 70% concurrent, 5 Mbps/user, 600 Mbps/AP, 50% util, 50 clients/AP → concurrent = 140; totalBw = 700 Mbps; effectiveAp = 300; apsByThroughput = ceil(700/300) = 3; apsByDensity = ceil(140/50) = 3; recommended = 3.

FIELD NOTES

- A planning model, not a survey. It ignores RF coverage, building layout, and co-channel interference, which often drive AP count higher than capacity math alone.
- "AP max throughput" should be a realistic per-AP usable rate, not a marketing PHY peak.
- Concurrency and per-user demand are the biggest assumptions; size them from the actual application mix.

Source: `capacity_planner_screen.dart`

PoE Budget

Checks a PoE switch's total power budget against the sum of connected device power draws, with an over/caution/OK verdict.

WHY IT'S HERE

Before connecting a batch of APs (and cameras, phones) to a switch, verify the switch can power them all without exceeding its PoE budget.

HOW TO USE

1. Enter the switch's PoE budget in watts.
2. For up to six device rows, enter watts-per-device and quantity (quantity defaults to 1).
3. Read total draw, remaining budget, percent used, and the verdict.

INPUTS

INPUT	UNIT	RANGE
Switch PoE budget	watts	must be > 0 (else no result)
Device rows (up to 6)	watts (decimal, blank → 0) and quantity (integer, default 1, blank → 0)	(:73)

FORMULA OR METHOD

> total_draw = Σ (watts · qty) across rows. remaining = budget − total; pct = min(100, total/budget · 100). Verdict order: remaining < 0 → OVER; else pct > 80 → CAUTION; else OK. (poe_budget_screen.dart, math section)

EXAMPLE

> Budget 370 W, six APs at 25.5 W each (qty 6) → total = 153 W; remaining = 217 W; pct = 41% → OK.

FIELD NOTES

- Use each device's worst-case (max) draw, not idle, and account for the PoE class.
- The pct is capped at 100 for display, but the OVER verdict triggers on actual negative remaining, so an over-budget case is never masked.
- This does not model per-port limits or cable-length power loss, only the switch's aggregate budget.

Source: poe_budget_screen.dart

Throughput Calculator

Computes the PHY rate and an estimated real throughput for a Wi-Fi connection from standard, channel width, MCS index, spatial streams, and guard interval.

WHY IT'S HERE

Translates a client's negotiated rate parameters into expected data-rate numbers, and sets realistic expectations for "what speed should I see."

HOW TO USE

1. Pick the Wi-Fi standard (802.11n/ac/ax/be).
2. Pick channel width, MCS index, spatial streams, and guard interval (the options reclamp to what's valid for the chosen standard).
3. Read modulation, PHY rate, and estimated real throughput in Mbps.

INPUTS

INPUT	UNIT	RANGE
Standard	select	HT (802.11n), VHT (802.11ac), HE (802.11ax, default), EHT (802.11be) (:44, :196)
Channel width	MHz	HT {20,40}; VHT/HE {20,40,80,160}; EHT {20,40,80,160,320} (:123-128)
MCS index	index	max per standard: HT 7, VHT 9, HE 11, EHT 13 (:107-112)
Spatial streams	count	up to: HT 4, VHT/HE/EHT 8 (:140-145)
Guard interval	µs	HT/VHT {0.4, 0.8}; HE/EHT {0.8, 1.6, 3.2} (:132-137)

FORMULA OR METHOD

> PHY_rate(Mbps) = (Nsd · bitsPerSymbol · streams) / symbolTime_µs (:153-167). Nsd = data subcarriers per standard per width (e.g. HE 80 MHz = 980) (:91-96). bitsPerSymbol = MCS_BPS[mcs] (Nbpsc·Rc), e.g. MCS 11 = 8.3333 (:52-67). symbolTime = OFDM symbol duration per standard per GI (e.g. HE @ 0.8 = 13.6 µs) (:99-104). real_rate(Mbps) = PHY_rate · efficiency, efficiency per standard: HT 0.70, VHT 0.72, HE 0.76, EHT 0.80 (:114-120, 172-189). Output rounded to 1 decimal. Invalid width/GI combo or out-of-range MCS → blank (:163-166). (throughput_calc_screen.dart:51-189)

EXAMPLE

> HE, 80 MHz, MCS 11, 2 streams, GI 0.8 → PHY = (980 · 8.3333 · 2) / 13.6 = 1201.0 Mbps; real = 1201.0 · 0.76 = 912.7 Mbps.

FIELD NOTES

- The efficiency factors (0.70 to 0.80) are flat per-standard estimates of MAC/overhead, not measured for your environment; real throughput depends on contention, retries, frame aggregation, and airtime sharing.
- PHY rate is the theoretical peak for a single, clean link. Treat the real-throughput number as an optimistic ceiling.

Source: `throughput_calc_screen.dart:51-189`

Coordinates & GPS 4

Distance and Bearing

Computes the great-circle distance and the initial (forward) and reverse bearings between two latitude/longitude points.

WHY IT'S HERE

Aiming point-to-point antennas. You need the distance (for FSPL) and the compass bearing (to point the dish) between two sites.

HOW TO USE

1. Enter latitude and longitude for point 1 and point 2 (decimal degrees).
2. Read distance (km), forward bearing, and reverse bearing.

INPUTS

INPUT	UNIT	RANGE
Point 1 / Point 2 latitude and longitude	decimal degrees	—

FORMULA OR METHOD

> Earth radius = 6371 km. Haversine distance: a = sin²(Δlat/2) + cos(lat1)·cos(lat2)·sin²(Δlon/2); d = 6371 · 2 · atan2(√a, √(1−a)). Forward bearing: y = sin(Δlon)·cos(lat2); x = cos(lat1)·sin(lat2) − sin(lat1)·cos(lat2)·cos(Δlon); bearing = (atan2(y, x)·180/π + 360) mod 360. Reverse bearing: (forward + 180) mod 360. (dist_bearing_screen.dart, math section)

EXAMPLE

> San Francisco (37.7749, −122.4194) to Los Angeles (34.0522, −118.2437) → distance = 559.12 km; forward bearing = 136.5°; reverse bearing = 316.5°.

FIELD NOTES

- Spherical earth model (6371 km mean radius), accurate to a fraction of a percent for terrestrial links, not for survey-grade geodesy.
- The forward bearing is the initial bearing of the great-circle path; it changes along the route on long paths.
- The reverse bearing is the simple +180°, which is exact only on a sphere.

Source: dist_bearing_screen.dart

Final Point

Computes the destination latitude/longitude given a starting point, an initial bearing, and a distance (the "direct" geodesic problem).

WHY IT'S HERE

Projecting where a link points. Given a site, a heading, and a distance, find the coordinates of the far end (e.g. to check a candidate tower location).

HOW TO USE

1. Enter the start latitude and longitude.
2. Enter the initial bearing in degrees.
3. Enter the distance and pick its unit (km, mi, or m).
4. Read the destination latitude and longitude.

INPUTS

INPUT	UNIT	RANGE
Start latitude / longitude	`decimal degrees`	—
Bearing	`degrees`	—
Distance	`km, mi, or m`	`mi × 1.60934 → km; m ÷ 1000 → km`

FORMULA OR METHOD

> `Earth radius = 6371 km; δ = dist_km / 6371 (angular distance), θ = bearing (radians). lat2 = asin(sin(lat1)·cos(δ) + cos(lat1)·sin(δ)·cos(θ)). lon2 = lon1 + atan2(sin(θ)·sin(δ)·cos(lat1), cos(δ) − sin(lat1)·sin(lat2)), longitude wrapped to (−180, 180]. (final_point_screen.dart, math section)`

EXAMPLE

> From (40, −105), bearing 90° (due east), distance 100 km → destination (39.99, −103.83), about 1.17° of longitude east at that latitude (latitude dips marginally because a constant-90° initial bearing follows a great circle, not a parallel).

FIELD NOTES

- Direct great-circle solution on a sphere; the bearing is the initial heading and the path curves on long distances.
- Same spherical-model accuracy caveats as the other geographic tools.

Source: `final_point_screen.dart`

Lat / Long Conversion

Converts a coordinate between decimal degrees (DD), degrees-decimal-minutes (DDM), and degrees-minutes-seconds (DMS).

WHY IT'S HERE

Different tools and maps use different coordinate notations; this normalizes between them when entering a site location.

HOW TO USE

1. Enter a latitude value (and/or longitude) in decimal degrees.
2. Read the DD, DDM, and DMS forms with the correct hemisphere letter.

INPUTS

INPUT	UNIT	RANGE
Latitude	decimal degrees	valid range ±90
Longitude	decimal degrees	valid range ±180

FORMULA OR METHOD

> Hemisphere: lat ≥ 0 → N else S; lon ≥ 0 → E else W (:66-74). Degrees = floor(|dd|); minutes = floor((|dd| − floor) · 60); seconds = remaining fraction · 60 (:77-92). Decimal minutes (DDM) = minutes + seconds/60 (:95-97). Format strings: DD to 6 decimals; DDM deg° min.mmmm' dir; DMS deg° min' sec.ss" dir (:101-118). Out-of-range or non-finite values blank the output. (lat_long_screen.dart:66-118)

EXAMPLE

> 40.7128° latitude → DD 40.712800; DMS 40° 42' 46.08" N; DDM 40° 42.7680' N.

FIELD NOTES

- Validates ranges (±90 lat, ±180 lon) and blanks on out-of-range input.
- The sign drives the hemisphere letter, so enter west longitudes and south latitudes as negative.
- No datum/projection handling; these are plain WGS84-style decimal degrees in, formatted out.

Source: `lat_long_screen.dart:66-118`

Midpoint

Computes the great-circle midpoint between two latitude/longitude points.

WHY IT'S HERE

Finding a relay or repeater site halfway along a long link, or the center point of a coverage area between two known locations.

HOW TO USE

1. Enter latitude and longitude for point 1 and point 2.
2. Read the midpoint latitude and longitude.

INPUTS

INPUT	UNIT	RANGE
Point 1 / Point 2 latitude and longitude	`decimal degrees`	—

FORMULA OR METHOD

> `Bx = cos(lat2)·cos(Δlon), By = cos(lat2)·sin(Δlon). lat_mid = atan2(sin(lat1)+sin(lat2), √((cos(lat1)+Bx)² + By²)). lon_mid = lon1 + atan2(By, cos(lat1)+Bx), then wrapped to (−180, 180] via ((deg + 540) mod 360) − 180. (midpoint_screen.dart, math section)`

EXAMPLE

> Between (40, 0) and (40, 90) the great-circle midpoint lies well north of the latitude average, at (49.88, 45.00), reflecting that the shortest path bows poleward.

FIELD NOTES

- This is the great-circle (spherical) midpoint, not the average of the coordinates. On east-west paths the midpoint is noticeably closer to the pole than the simple lat/lon average.
- Spherical model, same accuracy caveats as Distance and Bearing.

Source: `midpoint_screen.dart`

Conversions 4

dBm / Watt Converter

Live two-way conversion between dBm, Watts, and milliwatts. Type in any one field and the other two update in real time.

WHY IT'S HERE

RF power lands in your lap in three units depending on the spec sheet, the regulator, or the radio. This converts between them so you don't reach for a calculator at the AP.

HOW TO USE

1. Type a value in any of the three fields (dBm, Watts, or Milliwatts); the other two recompute instantly.
2. dBm and Watts accept scientific notation (e.g. 1e-10) plus a sign; Milliwatts is unsigned decimal.
3. Watts renders in scientific notation (5 sig figs) because real Wi-Fi receive levels are tiny (e.g. 1e-10 W); mW shows 4 fixed decimals.
4. Entering Watts or mW that are zero or negative shows "—" in the dBm field, since log10 of a non-positive number is undefined.

INPUTS

INPUT	UNIT	RANGE
dBm	dBm (decibel-milliwatts)	any real number; signed + scientific notation accepted
Watts	W	> 0 to convert to dBm; ≤ 0 yields —; signed + scientific notation accepted
Milliwatts	mW	> 0 to convert to dBm; ≤ 0 yields —; unsigned decimal

FORMULA OR METHOD

> Three pure functions (dbm_watt_converter.dart:72-74): dBm→W = 10^(dBm/10) / 1000; dBm→mW = 10^(dBm/10); W→dBm = 10 · log10(W · 1000). mW→dBm reuses W→dBm via mW/1000 (:120). The 1000 factor is the W↔mW conversion (1 W = 1000 mW); the in-app formula card states dBm = 10 · log10(mW) and W = 10^(dBm/10) / 1000 (:287-292). Watts formatted as toStringAsExponential(4) = 5 sig figs (:136-140); mW and dBm as toStringAsFixed (mW 4 decimals, dBm 2 decimals) (:104-105, 142-145). Non-positive W or mW → "—" in dBm because log10 is undefined (:95-99, :111-115). Matches the rf-tools-pwa app.js convertDbmToW / convertWToDbm / convertMwToDbm.

EXAMPLE

> 0 dBm → 10^(0/10) = 1 mW = 1e-3 W. +30 dBm → 10^(30/10) = 1000 mW = 1 W. 20 mW → 10 · log10(20) = 13.01 dBm.

FIELD NOTES

- 0 dBm is exactly 1 mW by definition, the reference point of the scale.
- The on-screen reference card anchors common values: +30 dBm = 1000 mW (1 W, FCC 2.4 GHz max conducted), +13 dBm = 20 mW (common default AP Tx power), 0 dBm = 1 mW, -67 dBm = 0.2 nW (minimum for enterprise data), -80 dBm = 10 pW (typical ambient noise floor).
- Watts uses scientific notation deliberately: Wi-Fi receive power sits around 1e-10 W, which fixed notation renders as an unreadable string of zeros.
- Formulas and behavior are ported verbatim from the rf-tools-pwa reference implementation (app.js).

Source: `dbm_watt_converter.dart:72-74,287-292`

Hex / ASCII

A live decimal/hexadecimal/binary integer converter plus a printable-ASCII reference table (codes 32 to 126).

WHY IT'S HERE

Reading packet captures, MAC/OUI bytes, and config hex values. Flip a number between bases or look up an ASCII character without leaving the app.

HOW TO USE

1. Type a value into any of the three converter fields (decimal, hexadecimal, binary); the other two update live.
2. Scroll the ASCII table to look up a printable character's decimal/hex/binary code.

INPUTS

INPUT	UNIT	RANGE
Decimal	`digits only`	—
Hexadecimal	`0-9, a-f, A-F (optional 0x prefix stripped)`	—
Binary	`0/1 (optional 0b prefix stripped)`	`Unsigned-integer domain, BigInt-backed, so arbitrarily long strings never overflow; a blank/invalid field blanks the mirrors`

FORMULA OR METHOD

> `Parsing/formatting via BigInt radix conversion (toRadixString(16), toRadixString(2), base-10). ASCII table: each row's char is derived at build time via String.fromCharCode(dec) — never hand-transcribed; hex is 2-digit uppercase, binary is 8-bit zero-padded. (hex_ascii_screen.dart, converter + AsciiRow)`

EXAMPLE

> Enter decimal 65 → hex 41, binary 1000001. In the table, decimal 65 = char A = hex 41 = binary 01000001.

FIELD NOTES

- Unsigned integers only: no negative numbers, no fractions, no two's-complement.
- The table covers printable ASCII (32 to 126) only; control characters and extended/Unicode code points are out of scope.
- The converter is base conversion, not text encoding (it converts a single integer, not an ASCII string to its byte sequence).

Source: `hex_ascii_screen.dart`

Metric Conversion

Converts a length between meters, kilometers, miles, feet, centimeters, inches, and nautical miles.

WHY IT'S HERE

Field work mixes metric and imperial constantly (datasheets in meters, tape measures in feet, link distances in miles). A quick unit converter avoids arithmetic slips.

HOW TO USE

1. Enter a value and pick its "from" unit.
2. Read the value in every other unit (the tool pivots through meters).

INPUTS

INPUT	UNIT	RANGE
Value and source unit	`one of m, km, mi, ft, cm, in, nmi`	—

FORMULA OR METHOD

```
Meters per unit: m = 1, km = 1000, mi = 1609.344, ft = 0.3048, cm = 0.01, in = 0.0254, nmi = 1852. convert(value, from, to) = (value · metersPerUnit[from]) / metersPerUnit[to]. Display decimals: m 4, km 6, mi 6, ft 4, cm 2, in 4, nmi 6. (metric_conversion_screen.dart, math section)
```

EXAMPLE

1 mi → 1609.344 m → 1.609344 km, 5280.0 ft, 0.868976 nmi.

FIELD NOTES

- Uses the international mile (1609.344 m) and international foot (0.3048 m), not the US survey foot.
- Conversions are exact within floating point; the per-unit decimal rounding is for display only.

Source: `metric_conversion_screen.dart`

Unit Converter

Converts a value between units in one of eight categories: data transfer rate, data storage, length, power, metric prefix, speed, temperature, and time.

WHY IT'S HERE

Field work mixes units constantly. A datasheet quotes throughput in Mbps, your test rig reports MB/s, an RF reading lands in dBm when you wanted milliwatts. Pick a category, pick the two units, read the answer.

HOW TO USE

1. Pick a category (data transfer rate, data storage, length, power, metric prefix, speed, temperature, or time).
2. Type a value, then pick the "from" unit and the "to" unit.
3. Read the converted result. It blanks until the value parses to a valid number.

INPUTS

INPUT	UNIT	RANGE
Category	one of eight categories	—
Value, from-unit, to-unit	the units offered in the chosen category	signed and scientific input accepted (a temperature can be negative)

FORMULA OR METHOD

> Linear categories convert through a single base unit: value × factor(from) ÷ factor(to). Power and temperature are not linear and route through their own helpers. Power treats dBm with the same log math as the dBm/Watt converter (W = 10^(dBm/10)/1000; dBm = 10·log10(W·1000)); temperature is affine through Kelvin. (unit_conversion.dart)

EXAMPLE

> 100 MB → 800 Mbit (storage, decimal). 100 °C → 212 °F (temperature). 1 W → 30 dBm (power).

FIELD NOTES

- Decimal and binary are kept separate. A KB is 1000 bytes; a KiB is 1024 bytes. The tool never conflates the two, so a storage answer reads true to the standard you picked.
- Bits and bytes are different units, not a display toggle. 1 byte = 8 bits, and the converter treats them that way across both the storage and the rate categories.
- Temperature and dBm are not simple multiply-by-a-factor conversions. They are handled with the correct affine and logarithmic math, so a negative temperature or a sub-milliwatt power reads correctly.

Source: `unit_converter_screen.dart`

Utilities & Generators 2

DTMF Generator

Plays the Touch-Tone keypad tones (0-9, *, #, and A-D) from a standard 4×4 DTMF grid, one tap at a time or as a continuous loop.

WHY IT'S HERE

Driving a system that still listens for Touch-Tones over the air or down a line: an IVR menu, a repeater controller, a legacy PBX, or a piece of test gear. Hold the device near the microphone and play the digit.

HOW TO USE

1. Tap a key to play its tone for a short burst. The selected key and its two frequencies show at the top.
2. Tap Play to loop the selected key's tone continuously; tap Stop to end it.
3. Tapping a different key during a loop retargets the loop to the new key.

INPUTS

INPUT	UNIT	RANGE
Keypad key	one of 1-9, 0, *, #, A-D	defaults to the center key, 5

FORMULA OR METHOD

> Each DTMF key is the sum of two sine waves, one low-group frequency (the row) and one high-group frequency (the column), per ITU-T Q.23. Low group: 697, 770, 852, 941 Hz. High group: 1209, 1336, 1477, 1633 Hz. A single key tap plays the pair for about 200 ms. Synthesis is local; nothing is dialed or transmitted by the app itself. (dtmf.dart, dtmf_generator_screen.dart)

EXAMPLE

> Key 5 is 770 Hz + 1336 Hz. Key 1 is 697 Hz + 1209 Hz. The * key is 941 Hz + 1209 Hz.

FIELD NOTES

- Every key is two tones added together, which is what "Dual-Tone Multi-Frequency" means. That is why the readout shows two frequencies, not one.
- The A, B, C, and D keys are real DTMF tones that never appeared on consumer phones. They show up in radio, military, and control systems, which is why the full 4×4 grid is here.
- Tones come out of the device speaker. The app generates audio; it does not place a call or send anything down a phone line. To control remote gear, play the tone into that system's microphone or audio input.
- Reliable detection depends on volume, the speaker, and the receiving system. Hold the device close and keep the level up if a stubborn IVR or controller misses a digit.

Source: dtmf_generator_screen.dart

QR Code Generator

Turns any text or URL you type into a scannable QR code, then lets you share or save it as an image.

WHY IT'S HERE

Handing off a config URL, a guest Wi-Fi link, or a site address without making someone type it. Generate the code on the spot and let them scan it with a phone camera.

HOW TO USE

1. Type the text or URL you want to encode.
2. The QR code renders live as you type.
3. Tap Share / Save to send the image through the system share sheet or save it.

INPUTS

INPUT	UNIT	RANGE
Text or URL	`any string`	`empty input shows a prompt and renders no code`

FORMULA OR METHOD

> `The code is generated locally with no network call. It renders as dark modules on a white background with a 4-module quiet zone, which is what scanners expect. Share / Save captures that white tile to a PNG. (qr_generator_screen.dart)`

EXAMPLE

> Type https://wlanpros.com and the matching QR code appears below the field, ready to scan or share.

FIELD NOTES

- Dark code on a white background, always. Inverted light-on-dark codes look on-brand but many scanners fail to read them, so the tool does not offer that option.
- The white border around the code is the quiet zone, and it is part of the code. Do not crop it out when you share the image, or the QR may not scan.
- Everything happens on the device. The text you encode never leaves the app except through the share sheet you trigger.
- Encodes plain text and URLs. It is a generator, not a scanner, so it makes codes rather than reading them.

Source: `qr_generator_screen.dart`

CATEGORY

Quick Reference

49 tools

Offline lookup tables and the laminated field cards. Channel plans, standards, thresholds, connector and cabling pinouts, protocol references, CLI and capture cheat sheets, checklists, and guides, all available without a connection.

Wi-Fi & RF 15

802.11 Standards

A PHY-layer comparison of every major 802.11 amendment from the original 802.11 (1997) through Wi-Fi 7, with year, bands, max PHY rate, MIMO, channel widths, and modulation.

WHY IT'S HERE

Settling "which generation does what": bands reached, max rate, MIMO ceiling, and modulation per amendment.

HOW TO USE

1. Scan by generation badge (Wi-Fi 4 through Wi-Fi 7).
2. The band filter answers "which generations reach 6 GHz" (Wi-Fi 6E and Wi-Fi 7).
3. The original 802.11 shows "—" for generation and MIMO (it predates both).

FIELD NOTES

- What it shows: one card per amendment with the IEEE designation, a Wi-Fi generation badge, year, and rows for Bands (GHz), Max PHY rate, MIMO, Channel width (MHz), and Modulation. An optional band filter (All / 2.4 / 5 / 6 GHz) narrows the list.
- Two footnotes ship in the code: (1) "Wi-Fi 1/2/3 are informal/retroactive labels; official Wi-Fi Alliance naming begins at Wi-Fi 4"; (2) "Wi-Fi 7 certification began 2024; IEEE 802.11be was published 2025."
- Max PHY rate is the theoretical aggregate ceiling; real-world throughput is typically 50 to 60% of it.
- Data source: IEEE 802.11 amendments; ported verbatim from the rf-tools-pwa STANDARDS const. Key rows: 802.11ac = Wi-Fi 5 (2013, 6.9 Gbps); 802.11ax = Wi-Fi 6 (2019) and Wi-Fi 6E (2021, adds 6 GHz); 802.11be = Wi-Fi 7 (2024, 46 Gbps MLO, 4K-QAM, up to 320 MHz).
- CATALOG NOTE: catalog id is 80211-standards, route /tools/standards.

Source: `lib/screens/tools/reference/standards_screen.dart`

Antenna Fundamentals

A read-along teaching reference for antenna literacy: what an antenna actually does (shapes where the radio's energy goes, it does not add power), azimuth vs elevation, why gain trades against beamwidth, polarization and the wall-clock mistake, downtilt, how to read a radiation-pattern polar plot (main lobe, the -3 dB beamwidth points, side lobes, nulls, front-to-back ratio), and which antenna type (omni, patch, sector, Yagi, dish) fits which space. Seven line diagrams are embedded at the points they teach.

WHY IT'S HERE

It is the antenna-literacy companion to the directional tools in the toolbox (AP Placement, Downtilt, Point-to-Point, Fresnel). When you are choosing or mounting an antenna and want to reason about coverage shape rather than chase a gain number, this is the read-first reference.

HOW TO USE

1. Read top to bottom: it builds from one idea (an antenna is a shaper, not a booster) through azimuth/elevation, orientation, reading a pattern chart, and what to use where.
2. Use the embedded diagrams alongside the prose: the polar-plot anatomy diagram labels every feature (main lobe, -3 dB beamwidth, side lobes, nulls, back lobe, front-to-back) so a manufacturer's chart stops being a mystery.
3. Section 4's type table and the closing deployment quick-map are the fast lookups: match the space you are covering to the antenna whose pattern fits it.

FIELD NOTES

- This is conceptual teaching copy, not a calculator and not a manufacturer spec sheet. It deliberately gives no formula for gain vs beamwidth: "more gain means a narrower beam" is always directionally true, but the exact beamwidth for a given gain depends on the specific antenna's design. Read the published beamwidth on the data sheet; do not try to back it out of the gain number.
- The per-type beamwidth figures (patch ~30-120 degrees, sector ~60-120, Yagi ~15-40, dish ~3-25) are RANGES across real products, not single specs. The antenna in your hand lands somewhere inside its band.
- The cross-polarization penalty is given conceptually: 90 degrees is the worst case (in theory total loss; in practice reflections leave some signal), and a wall-mounted omni costs roughly 6 dB. These are rules-of-thumb to reason with, not an exact loss figure.
- The floor-plan coverage shapes are illustrative of how a pattern fills a space, not survey predictions or measured coverage.
- The point-to-point note is one line by design: a link needs real Fresnel-zone clearance around the straight line between antennas, not just a visible path. Use the Fresnel and Point-to-Point tools to size that clearance.

Source: Penn teaching copy (SOP-020 PASS, 2026-06-05), from Pax's research brief; diagrams by Charta. Rendered verbatim in antenna_fundamentals_screen.dart.

AP Placement

Field-tested design rules for AP location, cell sizing and overlap, channel planning, and high-density venues.

WHY IT'S HERE

A pre-survey checklist of placement do's and don'ts: mounting, spacing, overlap targets, and density ceilings.

HOW TO USE

1. Read top to bottom as guidance; each bullet is a complete recommendation.
2. Coverage radii given are starting points (20 to 30 m open office, 10 to 15 m walled), not guarantees.

FIELD NOTES

- What it shows: five rule groups, each a heading over a bulleted list: Start with requirements, AP location, Cell sizing and overlap (≥ 2 APs at −70 dBm everywhere, 15 to 20% overlap, typical coverage radii), Channel planning (2.4 GHz only 1/6/11, co-channel spacing, prefer 5/6 GHz, DFS), and High-density venues (reduce power add APs, directional antennas, 20 to 30 clients/radio ceiling, tri-radio caveats).
- The intro says coverage radii and spacing are starting points; validate every design with a post-installation survey.
- The code is explicit that an AP is never called a router. Per-radio capacity is a model-dependent ceiling.
- Data source: field-practice guidance; ported verbatim from the rf-tools-pwa aplace tool (AP_RULES). Not a single named standard, but accumulated WLAN design best practice.

Source: `lib/screens/tools/reference/ap_placement_screen.dart`

Channel Map

A visual channel-bonding map showing, per band, how 20/40/80/160/320 MHz channels bond together, which primary (center) channel labels each bonded block, and which blocks require DFS.

WHY IT'S HERE

When planning channel widths and bonding, to see at a glance which 20 MHz primaries fold into a given 80 or 160 MHz channel, and which bonds drag in a DFS sub-channel.

HOW TO USE

1. Scroll the 5/6 GHz maps horizontally. A block's number is its primary (center) channel.
2. Color/chip legend: No DFS (neutral, an attribute not a verdict), DFS (amber, radar detection required), Mixed / DFS (danger, a 160 MHz bond spanning DFS and non-DFS sub-bands, where any DFS sub-channel subjects the whole bond to DFS), PSC (lime, the 6 GHz preferred scanning channels).
3. The 6 GHz 320 MHz row shows ch 31 as the primary block and ch 63 as a dashed alternative (they overlap, so only one is used at a time).

FIELD NOTES

- What it shows: a three-option band toggle. 2.4 GHz: the 11 US channels as 20 MHz blocks, with 1/6/11 emphasized (non-overlapping) and the rest faint. 5 GHz: rows for 20/40/80/160 MHz bonded widths, each block labelled with its primary/center channel and tinted by DFS class. 6 GHz: rows for 20/40/80/160/320 MHz across UNII-5 (ch 1 to 93), with PSC channels marked.
- US-default. The full US 6 GHz band extends to ch 233 (UNII-6/7/8 follow the same bonding pattern); the map shows UNII-5 only. 6 GHz UNII-5 needs no DFS and no AFC indoors (LPI).
- Data source: US (FCC). Ported verbatim from the rf-tools-pwa chanmap tool. 5 GHz DFS: No DFS = UNII-1 (36 to 48) and UNII-3 (149 to 165); DFS = UNII-2A/2C. The original PWA's literal hex colors were re-expressed in the design-system status palette; the meaning (DFS/PSC/mixed) is preserved.

Source: `lib/screens/tools/reference/channel_map_screen.dart`

dB Reference

A decibel reference card: dB change → power/voltage ratio with rules of thumb, and common dBm anchor values with their power and real-world context.

WHY IT'S HERE

Quick mental math in the field: "+3 dB is double power," "what power is +30 dBm," "what's the FCC 2.4 GHz limit in dBm."

HOW TO USE

1. In the ratio table, positive gains render in lime, losses in red.
2. Key anchors: +3 dB = 2× power; +10 dB = 10×; +30 dBm = 1 W; 0 dBm = 1 mW; −67 dBm = enterprise VoIP minimum; −70 dBm = enterprise data minimum; −80 dBm = typical Wi-Fi receiver sensitivity.

FIELD NOTES

- What it shows: dB Power Ratios: dB change (+3 to +30, −3 to −20) → power ratio, voltage ratio, and a rule-of-thumb note. Common dBm Reference Points: dBm anchors (+36 down to −100 dBm) → power (watts/mW/nW/pW) and context (regulatory limits, typical Tx powers, sensitivity floors).
- Footnote: "0 dBd is about 2.15 dBi (dipole reference). dBW = dBm − 30. Regulatory limits shown are US FCC; verify before compliance decisions."
- Mixed US (FCC) and one ETSI anchor; read the context cell for the jurisdiction of each limit.
- Data source: ported verbatim from the rf-tools-pwa dbref tool (DB_RATIOS, DBM_REFS). The dBm context column cites specific regulatory limits: FCC 6 GHz standard-power EIRP (+36 dBm, AFC required), FCC 2.4 GHz max conducted (Part 15.247, +30 dBm), FCC UNII-2A/2C and UNII-1 conducted maxes, and ETSI 5 GHz EIRP (EN 301 893, +23 dBm).

Source: `lib/screens/tools/reference/db_reference_screen.dart`

MCS Index

Look up the modulation, coding rate, and PHY data rate for any 802.11 MCS index across channel widths, for 802.11n (HT), 802.11ac (VHT), and 802.11ax (HE), scaled by spatial-stream count.

WHY IT'S HERE

When you see an MCS index in a capture or client stats and want to know the modulation/coding behind it and the rate it should deliver at a given width and stream count.

HOW TO USE

1. Choose the standard (n / ac / ax) and stream count (1 to 8); rates update (rate = per-stream value × streams).
2. MCS 0 (BPSK 1/2) is the most robust/slowest; higher MCS indices use denser modulation (up to 1024-QAM for ax) for higher rates needing better signal.
3. Cells shown as "N/A" are genuinely invalid combinations, not zero or fabricated.

FIELD NOTES

- What it shows: two selectors, 802.11 standard (n / ac / ax) and spatial streams (1 to 8). Columns per standard: 802.11n = 20 LGI, 20 SGI, 40 LGI, 40 SGI; 802.11ac = 20/40/80/160 SGI; 802.11ax = 20/40/80/160 MHz (800 ns GI).
- 802.11ac MCS 9 is invalid at 20 and 40 MHz for a single stream (shown "N/A").
- Guard-interval definitions: 802.11n LGI = 800 ns / SGI = 400 ns; 802.11ac uses SGI; 802.11ax uses 800 ns GI.
- The notes card states actual throughput is typically 50 to 65% of the PHY rate. Rates are PHY-layer maximums, not delivered throughput.
- Data source: verbatim port of the rf-tools-pwa mcs tool (MCS_N / MCS_AC / MCS_AX), reflecting the IEEE 802.11n/ac/ax PHY rate tables.

Source: `lib/screens/tools/reference/mcs_index_screen.dart`

Non-Wi-Fi Wireless Channels

Look up the channel/frequency plans of the common non-Wi-Fi radios that share or sit beside the bands a Wi-Fi pro works in: LoRaWAN, IEEE 802.15.4, Bluetooth Classic, Bluetooth LE, and Zigbee.

WHY IT'S HERE

When you're chasing interference or co-existence in the 2.4 GHz ISM band, or sizing a sub-GHz IoT deployment, and need to know where these radios actually sit.

HOW TO USE

1. Each technology is its own section.
2. LoRaWAN plans flagged with a "verify" chip are version-dependent or sparsely sourced.
3. BLE rows are ordered by physical frequency (not index) so you can see how the 3 advertising channels interleave; an "Advertising" chip marks channels 37/38/39 (2402/2426/2480 MHz).
4. Zigbee's "common picks" (11, 15, 20, 25, 26) are convention, not a mandate.

FIELD NOTES

- What it shows: one card per technology. LoRaWAN: regional plan (EU868, US915, AU915, AS923, IN865, KR920, CN470, CN779, RU864), frequency range in MHz, and a channel-plan description. IEEE 802.15.4: band (868 MHz / 915 MHz / 2.4 GHz), channel-number range, spacing, center summary, region. Bluetooth Classic (BR/EDR): 79 channels, 1 MHz spacing, f = 2402 + k MHz, 2402 to 2480 MHz, ~1600 hops/sec, global. Bluetooth LE: all 40 channels in physical-frequency order with index, frequency in MHz, kind (Advertising / Data). Zigbee: 2.4 GHz 802.15.4 ch 11 to 26, sub-GHz bands, common 2.4 GHz channel picks.
- Bluetooth, BLE, and 802.15.4 use globally-fixed channel grids. LoRaWAN frequency plans are entirely region-defined; there is no global LoRaWAN channel map.
- Plans marked "verify" (CN470 version-dependent; CN779 deprecated/limited; RU864 sparsely sourced) must be confirmed against RP002 §2 and the local regulator.
- The BLE table uses an explicit piecewise frequency lookup, NOT a naive linear formula (the code notes the linear "2402 + 2·index" approach is a common BLE chart bug).
- 802.15.4's 868/915 MHz bands are region-restricted; 2.4 GHz ch 11 to 26 are global.
- Data source: Pax's verified research brief (Deliverables/2026-06-02-wireless-channels-reference/data-brief.md), cross-checked against LoRa Alliance RP002, IEEE 802.15.4, Bluetooth SIG Core Spec, and CSA/Zigbee spec. 802.15.4 centers verified against the standard formula (ch 0 = 868.3 MHz; ch 1 to 10 = 906 + 2·(k−1); ch 11 to 26 = 2405 + 5·(k−11)).

Source: `lib/screens/tools/reference/non_wifi_channels_screen.dart`

PoE Reference

Power-over-Ethernet reference: the 802.3 PoE standards (PSE/PD power, powered pairs, class range) and the PD power classes (0 to 8) with max power at the device.

WHY IT'S HERE

When sizing PoE, confirm what a switch port delivers vs what reaches the device, and which 802.3 standard / class a given AP needs.

HOW TO USE

1. PSE power is supplied at the switch; PD power is what's left at the device after cable loss (e.g. 802.3at supplies 30 W PSE, delivers 25.5 W PD).
2. The class table maps a PD's negotiated class to its max draw (e.g. Class 4 = 25.5 W = PoE+ max; Class 8 = 71.3 W = Type 4 max).

FIELD NOTES

- What it shows: PoE standards: 802.3af (PoE), 802.3at (PoE+), 802.3bt Type 3 (PoE++/4PPoE), 802.3bt Type 4 (PoE++ Hi), each with PSE watts, PD watts, powered pairs (2 of 4 or 4 of 4), and supported class range. PD power classes: class 0 to 8 → max power at the PD, the 802.3 standard defining it, and a note.
- Footnote points to the PoE Budget tool for sizing a switch against connected devices.
- Written "802.3" (not "802.3x"). Standard reference, not region-specific.
- Data source: IEEE 802.3af/at/bt; ported verbatim from the rf-tools-pwa poe tool (POE_STDS, POE_CLASSES).

Source: `lib/screens/tools/reference/poe_reference_screen.dart`

Roaming Parameters

The 802.11k/r/v fast-roaming protocols (what each does, what it requires) plus RSSI/SNR/latency design thresholds for enterprise roaming.

WHY IT'S HERE

When designing or troubleshooting roaming for VoIP/UC, confirming the protocol roles and the signal-overlap targets that make handoffs work.

HOW TO USE

1. The protocol heading shows the designation (lime) and full name.
2. In the thresholds table, the scenario word is status-tinted with a dot: green = good (the two design targets), amber = marginal (the overlap zone), red = bad (sticky-client trigger and unusable).
3. Design rules carry the actionable target (e.g. "≥ 2 APs at −67 dBm everywhere," "15 to 20% cell overlap minimum").

FIELD NOTES

- What it shows: Protocols block: 802.11r (Fast BSS Transition), 802.11k (Neighbor Report), 802.11v (BSS Transition Management), each with what it does, deployment requirements, and a field note. Thresholds block: five scenarios (VoIP/UC design target, standard data design target, roaming overlap zone, sticky-client trigger, unusable) each with min RSSI, min SNR, roam latency, design rule, and a status verdict.
- The intro states targets vary by client hardware and AP vendor: design guidelines, not guarantees.
- Field notes flag real client behavior: some legacy clients have 802.11r compatibility issues; Android/iOS generally honor 802.11v but some Windows drivers ignore BSS-TM entirely.
- Data source: IEEE 802.11k/r/v; ported verbatim from the rf-tools-pwa roaming tool (ROAMING_PROTOCOLS, ROAMING_THRESHOLDS). The code notes the PWA defines exactly these three protocols; OKC is not in the source and was deliberately not added.

Source: `lib/screens/tools/reference/roaming_screen.dart`

Signal Thresholds

RSSI and SNR targets: a quality scale for RSSI, minimum RSSI/SNR by application, and the SNR needed to reach each typical MCS index.

WHY IT'S HERE

When validating a survey or troubleshooting a complaint: "is −68 dBm good enough for VoIP?" or "what SNR do I need to hold MCS 7?"

HOW TO USE

1. RSSI bands carry a colored verdict dot plus the quality word (green = good, amber = marginal/Fair, red = bad/Weak/Poor); the word always carries the meaning, never color alone.
2. For the application table, read across to the min RSSI and min SNR your target use case needs.
3. The SNR→MCS table tells you the SNR floor to sustain a given rate.

FIELD NOTES

- What it shows: three blocks. RSSI quality scale: Excellent (> −50 dBm) / Good (−50 to −67) / Fair (−67 to −70) / Weak (−70 to −80) / Poor (< −80). Minimum signal by application: VoIP/real-time, HD video, general browsing, email/basic data, IoT/low-rate, and location/RTLS, each with min RSSI, min SNR, and a note. SNR to MCS (80 MHz, 1 SS): the SNR floor for each MCS index (MCS 0 at 5 dB up to MCS 11 at 35 dB) with an indicative rate.
- The intro explicitly states: values vary by client hardware, environment, and AP vendor; treat as field-planning guidelines, not guarantees.
- The SNR→MCS rates are indicative (80 MHz, single stream); MCS 10/11 rates are noted as 802.11ax.
- Data source: ported verbatim from the rf-tools-pwa rssi tool. These are field-planning reference thresholds, not derived from a single named standard.

Source: `lib/screens/tools/reference/signal_thresholds_screen.dart`

Spectrum Reference

A per-band fact sheet for the three Wi-Fi bands: total usable spectrum, supported standards, channel counts, non-overlapping counts, channel widths, DFS/radar requirements, common co-existing interferers, and key deployment notes.

WHY IT'S HERE

The one-screen "tell me everything about this band" reference: what lives in it, what interferes with it, what the power/DFS rules are.

HOW TO USE

1. Pick a band (2.4 / 5 / 6 GHz); read the eight facts top to bottom.
2. The colored range badge is decorative chrome paired with the band label (2.4 GHz = amber/congested, 5 GHz = blue, 6 GHz = green).
3. The "Co-existence" row names the band's common interferers or managed incumbents.

FIELD NOTES

- What it shows: each band shows a range badge plus eight key/value facts: Total spectrum, Standards, Channels (US), Non-overlapping, Channel widths, DFS / Radar, Co-existence, and Key notes.
- US-default; the footnote on every band reads "US (FCC) regulatory domain. Verify local rules before deployment."
- Carries useful specifics: UNII-1 indoor-only in some regions; UNII-2A/2C DFS implies a 60-second channel-availability delay after radar detection; 6 GHz has three US power modes, namely Standard Power (up to 36 dBm EIRP, AFC outdoors), LPI (up to 30 dBm, no AFC), and VLP (up to 14 dBm EIRP, no AFC, mobile); WPA3 mandatory on 6 GHz.
- Data source: US (FCC) regulatory domain. Ported verbatim from the rf-tools-pwa spectrum tool. Values: 2.4 GHz = 84 MHz (US); 5 GHz = ~580 MHz usable (UNII-1/2A/2C/3); 6 GHz = 1200 MHz (5925 to 7125 MHz).

Source: `lib/screens/tools/reference/spectrum_screen.dart`

Wi-Fi Authentication Glossary

Plain-language definitions of the Wi-Fi authentication terms, 58 of them, that a network or security pro actually meets: the 802.1X / EAP framework, RADIUS and AAA, WPA2 / WPA3 and SAE, PSK and Enterprise modes, certificates, and the supporting acronyms. Each entry pairs the full term and its abbreviation with a definition written for working engineers, grouped by topic.

WHY IT'S HERE

When a term in a config screen, a log, or a standards document is the thing standing between you and understanding the authentication flow. It answers what AAA, SAE, PMF, or EAP-TLS actually mean in Wi-Fi terms, without a vendor's slant.

HOW TO USE

1. Browse the terms grouped by category (Core Authentication, EAP methods, key management, and the rest).
2. Type in the search box to filter live across the term name, abbreviation, and definition.
3. Each entry shows the full term, its abbreviation when it has one, and a definition in Keith's voice.
4. Use the copy action to grab the current view (the filtered subset when searching, otherwise the full list) as plain text.

FIELD NOTES

- Vendor-neutral by design. Definitions describe the standards-based behavior, not one vendor's implementation.
- This is the authentication-focused sibling of the general Wi-Fi Glossary; it reuses the same data-driven, searchable, grouped screen with a different bundled dataset.
- Data source: the curated Wi-Fi Authentication Glossary edition (Deliverables/2026-06-05-wifi-auth-glossary), 58 terms. The bundled JSON is the source of truth.

Source: `lib/screens/tools/reference/wifi_glossary_screen.dart (assetPath: assets/data/wifi_auth_glossary.json)`

Wi-Fi Channels

Look up Wi-Fi channel numbers, center frequencies, channel widths, and DFS status across all four Wi-Fi bands: 2.4 GHz, 5 GHz, 6 GHz, and sub-1 GHz Wi-Fi HaLow.

WHY IT'S HERE

When you need to confirm a channel's center frequency, whether it requires DFS, whether it's a Preferred Scanning Channel, or whether a 2.4 GHz channel is even legal in your region.

HOW TO USE

1. Pick a band from the selector (2.4 / 5 / 6 GHz / HaLow).
2. 2.4 GHz: a "Non-overlap" chip marks channels 1, 6, 11 (the only non-overlapping 20 MHz channels in the US); a neutral "EU"/"JP" chip marks channels 12 to 14 as not US-usable.
3. 5 GHz: an amber "DFS" chip means radar detection is required (UNII-2A/2C); UNII-1/UNII-3 carry no DFS.
4. 6 GHz: every shown channel is a PSC (clients scan these first).
5. HaLow: the per-region table is the headline; only the US scheme is shown per-channel.

FIELD NOTES

- What it shows: four-option band selector. 2.4 GHz: channel (1 to 14), center GHz, occupied range in MHz (±11 MHz of the 20 MHz channel), non-overlapping channels (1/6/11) or the regulatory domain. 5 GHz: channel (36 to 165), center GHz, UNII sub-band (UNII-1 / 2A / 2C / 3), and a DFS flag. 6 GHz: the 15 Preferred Scanning Channels (PSC), each with center GHz. HaLow (802.11ah): three stacked tables, namely US 902 to 928 MHz 1 MHz channels (odd 1 to 51, center = 902.5 + 0.5 × (ch−1) MHz), US channel-width blocks (1/2/4/8/16 MHz), and per-region operating ranges.
- US-default throughout. 2.4 GHz footnote: EU adds 12 to 13, JP adds 14. 5 GHz footnote warns to verify UNII-4 (ch 169 to 177) local rules before use. 6 GHz full band is 59 × 20 MHz (ch 1 to 233); indoor/LPI needs no AFC, outdoor/standard-power needs AFC; Wi-Fi 7 adds 320 MHz channels.
- HaLow is region-dependent and only the US scheme is verified per-channel; other regions show operating ranges only, marked "region-dependent," and China is explicitly flagged UNCERTAIN ("varies, reported 755 to 787, confirm with CMIIT").
- The code notes a faulty 930.5 MHz HaLow extraction was rejected by a band-edge check; ch 51 = 927.5 MHz.
- Data source / standard: US (FCC) regulatory by default. 6 GHz centers and HaLow centers derive from IEEE 802.11ax / 802.11ah formulas; ported from the rf-tools-pwa channels tool. 2.4 GHz channel 14 uses the special 2484 MHz (JP) step.

Source: `lib/screens/tools/reference/wifi_channels_screen.dart`

Wi-Fi Tools Comparison

A vendor-neutral, offline reference that compares professional Wi-Fi survey, design, spectrum-analysis, and troubleshooting toolkits, grouped by the activity each one serves. Each config lists its vendor, product, license model, up-front cost, and 3-year total cost of ownership, with a neutral note on what the bundle does and does not include. A per-vendor summary and a typical-toolkit roll-up ride alongside.

WHY IT'S HERE

There is no neutral, capability-level map of the professional Wi-Fi tooling landscape. This is that map, built from Keith's vendor-interviewed workbook, so a leveling-up engineer can see the four activities and which tools serve which job without a vendor sales pitch.

HOW TO USE

1. Browse by activity: Design, Validation, Spectrum Analysis, then Troubleshooting. Within each activity, configs are listed alphabetically by vendor.
2. Search by vendor (e.g. Ekahau), product, activity (e.g. spectrum), capability (e.g. survey), or license model (e.g. perpetual). The match is a case-insensitive substring and narrows the activities in place.
3. Read the up-front and 3-year TCO figures as planning estimates, not quotes. Use the vendor Website and Docs links to confirm current pricing and product details before you buy.
4. A query that matches nothing shows an honest "No match" card; it never fabricates a tool. The typical-toolkit roll-up and the per-vendor summaries are always available below the activities.

FIELD NOTES

- This comparison is in BETA REVIEW. Vendors are being consulted on the figures, and a few may change before the final release.
- Pricing is dated. The figures are as of February 2026; confirm current pricing with each vendor, because prices, bundles, and product names change often.
- Cost figures are MODELED ESTIMATES assembled by WLAN Pros from vendor-supplied numbers and list pricing, not vendor-published quotes. Treat every dollar amount as a planning estimate, never a binding price.
- This is NOT a ranking. There is no score and no "best". Tools are listed alphabetically and the same vendor can appear under more than one activity, because real toolkits are assembled across vendors. The set reflects vendors interviewed by WLAN Pros, not every tool that exists, so an omitted tool is not a snub.
- No vendor logos or product photos appear here. Those are trademarks and copyrighted images that need each vendor's written permission, which is still being gathered. The reference is text and data only for now.
- Fully offline: the comparison is a bundled asset (assets/data/wifi_tools_comparison.json) parsed once at screen open; malformed rows are skipped, not rendered as blanks. To add or correct an entry, edit that file.
- Source: the Wi-Fi Design, Validation, Spectrum Analysis & Troubleshooting Tools V6 workbook (Keith Parsons, vendor-interviewed, last revised 2026-02-16), with neutral writeups from the Pax research brief, 2026-06-05.

Source: `lib/screens/tools/reference/wifi_tools_comparison_screen.dart`

WPA Security

A matrix of Wi-Fi security modes (WEP through WPA3-Enterprise) with encryption, key method, PMF support, and a deployment verdict, plus a reference of the advanced features that distinguish them.

WHY IT'S HERE

When choosing or auditing an SSID's security, confirm a mode's cipher, key method, PMF requirement, and whether it's recommended, acceptable, or to be avoided.

HOW TO USE

1. Each mode shows a verdict chip whose color reflects the deployment recommendation, always with the word: WEP/WPA1 = "Do not deploy"/"Deprecated" (red); WPA2-Personal = "Acceptable" (amber); WPA3-Personal/WPA3-Enterprise = "Recommended" (green); Enhanced Open = "Open networks", WPA2-Enterprise = "Enterprise std" (blue/info).
2. Below, the feature rows explain the mechanisms (e.g. SAE replaces the PSK 4-way handshake with forward secrecy; OWE encrypts open networks without a password; 6 GHz requires WPA3 or OWE).

FIELD NOTES

- What it shows: Security modes: WEP, WPA (WPA1), WPA2-Personal, WPA3-Personal, Enhanced Open, WPA2-Enterprise, WPA3-Enterprise, each with category, encryption suite, key method, PMF (Not supported / Optional / Required), and a status verdict chip. Advanced features: SAE, PMF (802.11w), OWE, Forward Secrecy, 192-bit Security Mode, WPA3-mandatory-on-6 GHz, and 802.1X/RADIUS roles.
- WPA3-Enterprise's 192-bit mode (GCMP-256 + HMAC-SHA-384 + ECDH/ECDSA-384) is noted as required for government/classified deployments. 6 GHz does not permit WPA2 or older.
- The verdicts are reference guidance reflecting current best practice.
- Data source: IEEE 802.11 / Wi-Fi Alliance security standards; ported verbatim from the rf-tools-pwa wpa tool (WPA_MODES, WPA_FEATURES). Verdict colors were remapped from the PWA's raw hues to the design-system status tokens to clear WCAG contrast.

Source: `lib/screens/tools/reference/wpa_security_screen.dart`

Cabling & Connectors 8

Antenna Connectors

An 18-connector practical reference for Wi-Fi antenna systems: each connector's full name, reverse-polarity variant, typical Wi-Fi use, indoor/outdoor fit, coupling mechanism, body size, RF signal path, impedance, frequency rating, mating compatibility, and field notes. Plus a polarity-explained diagram, an at-a-glance comparison table, enterprise vendor trends, a to-scale size comparison, and the top 6 a Wi-Fi engineer actually meets.

WHY IT'S HERE

When you are identifying or mating an antenna lead in the field. It answers the questions a tech actually asks: is this RP-SMA or SMA, will these two mate, does it cover 6 GHz, and which connector does this vendor ship.

HOW TO USE

1. Browse the connectors grouped by where they live (enterprise panel/external, outdoor/point-to-point, board-level/internal, test/cellular). Each card shows the connector name, full name, an RP chip when it is a reverse-polarity variant, and a real photo where a freely-licensed one exists (some connectors keep a line diagram instead).
2. Each card lists typical use, indoor/outdoor, coupling, size, RF path, impedance, frequency, and mating, then a field-notes line.
3. Type in the search box to filter live across every field (name, vendor, coupling, size, RF path, frequency, or any word in a note).
4. Below the table: a polarity-explained diagram, a Compare-at-a-glance table (connector, size, RF path, typical use) that scrolls sideways on a phone, enterprise vendor trends, the size order largest to smallest with a to-scale diagram, and the top 6 connectors most engineers meet.

FIELD NOTES

- The key field gotcha: every connector here is 50 ohm, and an RP connector will thread onto its standard counterpart but the center contacts will not connect.
- U.FL intermates with I-PEX MHF I (same footprint); the smaller MHF 4 does NOT mate with either.
- DART is Cisco's Smart Antenna Connector, a proprietary multi-port interface that breaks out to RP-TNC, N-type or RP-N via Cisco adapter cables.
- Size is the connector body width (across-flats for threaded parts, outer diameter for board-level parts) — an approximate field-recognition aid, not a precision mechanical spec.
- Connector photos are shown only where a freely-licensed (CC0/public-domain) photo exists. N-Type, TNC and RP-TNC currently have no such photo, so they keep a line diagram rather than a stand-in.

Source: Keith Parsons draft, verified and augmented by Pax (2026-06-05).

Coax Cable

A coaxial cable reference: impedance, velocity factor, outer diameter, maximum usable frequency, and typical use for common RG- and LMR-series cables.

WHY IT'S HERE

When specifying an antenna run, pick the right LMR size for the length and frequency, and avoid a 75 Ω mismatch.

HOW TO USE

1. Each cable is a block: name, then a spec line (impedance / VF% / diameter / max GHz), then the use note.
2. The 75 Ω entry (RG-6) renders dimmed: it's impedance-mismatched for 50 Ω Wi-Fi and shown for reference only ("CATV/satellite, NOT for Wi-Fi").
3. Higher VF means slightly lower propagation delay and loss.

FIELD NOTES

- What it shows: per cable (RG-58, RG-8/U, RG-213, RG-214, LMR-100A through LMR-1200, RG-6): impedance, velocity factor (%), diameter (mm), max frequency (GHz), and a typical-use note.
- Footnote points to the Cable Loss tool for exact attenuation. Wi-Fi is a 50 Ω system, so the dimmed 75 Ω RG-6 is a mismatch. Max frequencies are typical maximums. Standard reference, not region-specific.
- Data source: ported verbatim from the rf-tools-pwa coax tool (COAX_DATA). Manufacturer/industry spec values for the named cable series.

Source: `lib/screens/tools/reference/coax_cable_screen.dart`

Ethernet Cable

Twisted-pair Ethernet cable categories (Cat5e through Cat8) with bandwidth, max speed, distance at 1G/10G, PoE support, shielding, and typical use.

WHY IT'S HERE

When choosing cable for a run, confirm a category's bandwidth, 10G reach, and PoE++ suitability (notably the Cat6A-for-PoE++ recommendation).

HOW TO USE

1. Scroll the table horizontally. "N/A" in a distance column means that rate isn't supported (e.g. Cat5e has no 10G distance).
2. Key facts: Cat6 hits 10G only to 55 m; Cat6A hits 10G to the full 100 m and supports all 802.3bt; Cat8 carries 1G/10G to 100 m but its 25/40G design rate is limited to ~30 m.

FIELD NOTES

- What it shows: per category (Cat5e, Cat6, Cat6A, Cat7, Cat7A, Cat8): max bandwidth (MHz), max speed, distance at 1 Gbps, distance at 10 Gbps, PoE support, shielding, and a typical-use note.
- Footnote PoE++ tip: bundled Cat6 running PoE++ generates significant heat; Cat6A dissipates it better; TIA-568 recommends Cat6A for PoE++ in bundles.
- Cat7/Cat7A use non-standard plugs ("Specialty"). The PWA's em-dash "not applicable" cells are rendered as ASCII "N/A" (no em dash). Standard reference, not region-specific.
- Data source: ported verbatim from the rf-tools-pwa ethernet tool (ETH_DATA); reflects the TIA-568 / ISO cabling categories. The footnote cites TIA-568's recommendation of Cat6A for PoE++ in cable bundles (heat dissipation).

Source: `lib/screens/tools/reference/ethernet_cable_screen.dart`

Ethernet Pinout

The T568A and T568B RJ-45 wiring standards: pin number, wire color, twisted pair, and 100/1000 Base-T signal function.

WHY IT'S HERE

When terminating or testing a cable, confirm which color goes on which pin for the chosen standard, and what each pin carries.

HOW TO USE

1. Pick the standard (T568B is the default and most common). View is "plug face, clip down, pin 1 on the left."
2. Striped wires show a split swatch (color over white). The swatch colors are real copper-pair colors (data, not UI theme).
3. T568A and T568B differ only in swapping the green and orange pairs.

FIELD NOTES

- What it shows: a T568B / T568A toggle. Each standard's eight pin rows show the pin number, a wire-color swatch and name (e.g. "Orange / White"), the twisted-pair number (1 to 4) with a pair-color swatch, and the 100/1000 Base-T function (TX+, RX–, BI-D A+, etc.).
- Footnote: applies to Cat5/5e/6/6A/7/8; a crossover cable uses T568A on one end and T568B on the other, rarely needed today since most switches/NICs auto-MDI-X. Standard reference, not region-specific.
- Data source: TIA-568 wiring standards (T568A/T568B); ported verbatim from the rf-tools-pwa pinout tool (PINOUT, pairColors).

Source: `lib/screens/tools/reference/ethernet_pinout_screen.dart`

Fiber Optic

Fiber types (OM1 to OM5, OS1/OS2) with core/cladding, modal bandwidth, jacket color code, and supported distance at 1G/10G/40G/100G.

WHY IT'S HERE

When specifying fiber, confirm a fiber type's reach at a given rate, its jacket color, and whether it's current or legacy.

HOW TO USE

1. Scroll the distance matrix horizontally. A "—" in a rate column means that rate isn't supported (OM1/OM2 don't do 40G/100G).
2. OM1/OM2 render dimmed (legacy). Jacket colors: OM1/OM2 = Orange, OM3 = Aqua, OM4 = Violet/Aqua, OM5 = Lime Green, OS1/OS2 = Yellow.
3. Multimode (OM) lists modal bandwidth; singlemode (OS) shows "N/A" bandwidth but reaches 10+ to 80+ km.

FIELD NOTES

- What it shows: two sub-tables. Distance by data rate, per fiber type: core/cladding, modal bandwidth (MHz·km), and supported distance at 1G/10G/40G/100G. Jacket color code & notes, per fiber type: a jacket color swatch and name, and a deployment note.
- Footnote: distances are per TIA-568/ISO 11801; actual limits depend on transceiver, splice count, and connector loss. OM3/OM4 are the current deployment standards; OM1/OM2 are legacy (dimmed). OM5's wideband window note (~1,850 to 2,470 MHz·km near 953 nm) is preserved verbatim. Standard reference, not region-specific.
- Data source / standard: TIA-568 / ISO 11801 (cited in the footnote); ported verbatim from the rf-tools-pwa fiber tool (FIBER_DATA).

Source: `lib/screens/tools/reference/fiber_optic_screen.dart`

Optical Transceivers

Searchable, offline reference of 35 optical Ethernet transceiver variants (1G to 400G) grouped by speed tier, plus the 9-row SFP-to-OSFP form-factor ladder. Each variant lists its designation, data rate, reach, fiber type, wavelength, and connector — with reach as the lead, trust-first column.

WHY IT'S HERE

Picking an optic, the field's first question is "which module, and how far will it go on this fiber?" This answers it without leaving the app or going online, and keeps the IEEE-vs-vendor line visible so you never quote a vendor reach as a standard guarantee.

HOW TO USE

1. Browse by speed tier. The commonly-ordered tiers (10G / 25G / 100G) surface first and are flagged "Commonly ordered"; 1G / 40G / 200G / 400G follow.
2. Search by designation (e.g. LR4), reach, fiber (SMF / MMF / OM4), wavelength (e.g. 850 nm), connector (LC / MPO), or tier (e.g. 100G). The match is a case-insensitive substring across all of those, and it narrows the tiers in place.
3. Read the REACH line first — it is the IEEE maximum on the listed fiber. The fiber chip (MMF / SMF) and connector chip (LC / MPO) give the physical-layer specifics at a glance.
4. A query that matches nothing shows an honest "No match" card; it never fabricates a module. The form-factor table is always available below the variants.

FIELD NOTES

- IEEE vs VENDOR: variants ratified in IEEE 802.3 carry a neutral IEEE chip. Vendor / coherent variants (1000BASE-EX, 1000BASE-ZX, 10GBASE-ZR, and 400GBASE-ZR) carry an amber VENDOR chip and a "loss-budget dependent" caveat. Their reach is real and widely sold but NOT an IEEE guarantee — it depends on the link's loss budget. Never quote a vendor reach as a standard figure.
- 400GBASE-ZR is coherent DWDM (OIF 400ZR), beyond base IEEE 802.3 — a metro / data-center-interconnect optic, not a base Ethernet PHY. Its 80-120 km reach is loss-budget and DWDM-line-system dependent.
- Reach figures are IEEE-standard maximums on the listed fiber; real vendor modules may differ. Multimode reach is grade-dependent (OM3 vs OM4 vs OM5), so those rows give per-grade numbers rather than one figure.
- What it contains: 35 verified variants spanning 1G to 400G across SFP, SFP+, SFP28, QSFP+, QSFP28, QSFP56, and QSFP-DD / OSFP, plus a 9-row form-factor ladder (max rate, lane count, typical power envelope). Power figures are typical vendor ranges, not a single standard — treat as guidance.
- Fully offline: the table is a bundled asset (assets/data/optical_transceivers.json) parsed once at screen open; malformed rows are skipped, not rendered as blanks. To add or correct an entry, edit that file.
- Verified against IEEE 802.3 standards tables (via Wikipedia's clause-mirroring pages) and Cisco / FS.com vendor datasheets (Pax verification brief, 2026-06-05). Out-of-scope bleeding edge (800G / 1.6T, CPO / LPO, SFP-DD) was deliberately excluded as not yet stable or field-relevant.

Source: `lib/screens/tools/reference/optical_transceivers_screen.dart`

RF Connectors

Common coaxial RF connectors (N, TNC, BNC, SMA, RP-SMA, MCX, MMCX, U.FL/IPEX, F-Type) with impedance, max frequency, mating style, and field notes.

WHY IT'S HERE

When matching a pigtail or antenna lead, confirm a connector's impedance, frequency ceiling, and the RP-SMA vs SMA / 50 Ω vs 75 Ω gotchas.

HOW TO USE

1. Each block shows the connector name with an impedance chip, then Max freq. and Mating as data rows, then notes.
2. The 50 Ω connectors read as a quiet neutral chip; the 75 Ω F-Type reads in the warning hue with a text flag marking it not for WLAN (the cue is text plus color, not color alone).
3. Notes carry the practical context (e.g. N-Type is the outdoor WLAN standard; RP-SMA has a reversed center pin and is NOT interchangeable with SMA; U.FL is fragile, ~30 mate cycles).

FIELD NOTES

- What it shows: per connector: name, impedance chip, max frequency, mating style, and a field-notes line. Rendered as stacked blocks (the original 5-column table is too wide for a phone).
- Footnote: Wi-Fi is a 50 Ω system, so F-Type (75 Ω) carries an impedance mismatch and significant loss; do not use on WLAN antenna runs. Frequency ranges are typical maximums, verify against the specific part. Standard reference, not region-specific.
- Data source: ported verbatim from the rf-tools-pwa rfconn tool (RF_CONN_DATA); industry connector specs.

Source: `lib/screens/tools/reference/rf_connectors_screen.dart`

RJ Connectors

A reference to the registered-jack connector form factors (RJ11, RJ14, RJ25, RJ45/8P8C, RJ48, RJ48C, RJ48X): positions, conductors, the modular body each uses, and its typical use. This is about the connector body, not the wiring.

WHY IT'S HERE

In the field you meet the same modular bodies for very different jobs: RJ11 carries one phone line, RJ45 (8P8C) carries Ethernet, and RJ48 reuses the 8P8C body for T1/E1 with a different pin assignment. Telling them apart keeps you from cabling the wrong jack.

HOW TO USE

1. Read each connector card: the name, its modular notation (e.g. 8P8C), positions, conductors, and typical use.
2. For T568A / T568B pin-to-pair-color wiring on the RJ45 (8P8C) connector, tap the cross-link card to open the Ethernet Pinout tool.

FIELD NOTES

- Accuracy: "RJ45" is the colloquial name for the 8-position 8-conductor (8P8C) modular connector used for Ethernet. Strictly, RJ45 was a telephone wiring standard; the data connector is properly the 8P8C modular jack.
- The PnCm notation means an n-position body with m conductors populated, e.g. 6P2C is a 6-position body with 2 conductors (RJ11), 8P8C is fully populated (RJ45/RJ48).
- This table does NOT duplicate the T568A/T568B pin colors: that wiring lives in the Ethernet Pinout tool, which this screen cross-links to.
- Data source / standard: the registered-jack (USOC) interface standards and the modular-connector form-factor conventions.

Source: `lib/screens/tools/reference/rj_connectors_screen.dart`

Protocols 7

802.11 Frame Exchange

The frame-by-frame sequences for common association and roaming scenarios, showing the order and direction of frames between the STA, AP, RADIUS server, and DHCP server.

WHY IT'S HERE

When analyzing a capture or explaining an association/roam, confirm what frame should come next and which entities exchange it.

HOW TO USE

1. Pick a scenario (Open / WPA2-PSK, WPA3-SAE, WPA2-Enterprise (802.1X/EAP), 802.11r Roam); read the phases top to bottom.
2. Each frame carries a neutral type code: MGMT (management frame), EAP (EAP / EAPOL key), WIRED (RADIUS/DHCP over the wire), DHCP.
3. The legend at the bottom expands each code. Type is carried by the text code, not by color.

FIELD NOTES

- What it shows: a scenario selector with four scenarios. Each scenario is broken into named phases (e.g. Probe & Auth, Association, 4-Way Handshake, EAP Authentication, DHCP); each frame shows a step number, direction, frame name, a frame-type code, and an explanatory note.
- These are representative sequences, not exhaustive; optional/passive-scan paths and EAP-method round-trips are summarized (e.g. the WPA3 DHCP phase is shown as one combined Discover/Offer/Request/Ack line "identical to WPA2 flow").
- Data source: IEEE 802.11 association/handshake behavior; ported verbatim from the rf-tools-pwa frames tool (FX_SCENARIOS). Reflects the real frame exchanges (e.g. SAE Dragonfly commit/confirm for WPA3, the 4-way handshake, 802.1X/EAP over RADIUS, 802.11r FT over-the-air). Standard reference, not region-specific.

Source: `lib/screens/tools/reference/frame_exchange_screen.dart`

802.11 Reason Codes

The 802.11 deauthentication/disassociation reason codes (RC) and association status codes (SC) that appear in captures, with a searchable filter.

WHY IT'S HERE

When a capture shows a deauth with reason code 15 or an association response with status 17, and you need the plain-language meaning fast.

HOW TO USE

1. Type a code number or keyword (e.g. "15" or "handshake") to filter; a "no match" card appears if nothing matches.
2. Reason codes (RC) appear in Deauthentication and Disassociation frames; status codes (SC) appear in Authentication, Association, and Reassociation Response frames.
3. Code 0 in the status group is the success value, rendered green.

FIELD NOTES

- What it shows: reason codes grouped by theme: Common (1 to 9), Capability/Channel mismatch (10 to 11), Security frame/element errors (13 to 14, 17 to 22, 24), Security handshake failures (15, 16, 23), QoS/load management (34 to 39), Fast Roaming/802.11r (45 to 48). Plus a separate Association Status Codes group (the most-common subset, 0 to 104), where code 0 ("Successful") is highlighted in green.
- The status-code list is the "most common" subset, not the full table.
- Codes and meanings are reproduced verbatim from the standard via the PWA source; nothing invented.
- Data source / standard: IEEE 802.11-2020 §9.4.1.7 (reason codes) and §9.4.1.9 (status codes), cited in the footnote. Codes and groupings ported verbatim from the rf-tools-pwa reason tool.

Source: `lib/screens/tools/reference/reason_codes_screen.dart`

HTTP Status Codes

The HTTP response status codes, grouped by class, with a plain-English meaning for each. A fast offline lookup when a captive portal, web service, proxy, or API returns a code and you need to know what it means.

WHY IT'S HERE

When a check returns 403 or 503, or a captive-portal probe comes back 511, and you want the meaning without leaving the toolbox or going online.

HOW TO USE

1. Type a code number or keyword (e.g. "404" or "redirect") to filter; a "no match" card appears if nothing matches.
2. The filter matches the code number, the reason phrase, and the plain-English meaning, so "timeout" finds 408 and 504.
3. Use the toolbar copy action to copy the full reference as tab-separated text, one section per class.

FIELD NOTES

- What it shows: codes grouped by class: 1xx Informational, 2xx Success, 3xx Redirection, 4xx Client Error, 5xx Server Error. Each row is the code number, its reason phrase, and a short meaning.
- 511 (Network Authentication Required) is the signature of a captive portal: the network blocks access until the client authenticates.
- 418 is registered in the IANA registry as "(Unused)". It is widely known as the "I am a teapot" joke code from RFC 2324; the tool labels it honestly and notes the history.
- Data source: the IANA HTTP Status Code Registry (the authoritative registry), fetched 2026-06-04. Most codes are defined by RFC 9110 (HTTP Semantics). Code numbers and reason phrases are verbatim from the registry; the plain-English meanings are written for this tool. Unassigned and obsoleted codes are omitted; nothing is invented.

Source: `lib/screens/tools/reference/http_status_codes_screen.dart`

OSI Model

The 7-layer OSI reference model: layer number, name, one-word function, PDU, example modern protocols, and typical hardware.

WHY IT'S HERE

Localizing a fault: which layer is failing tells you which tool to reach for.

HOW TO USE

1. Scroll the table horizontally. Read by layer number.
2. Example mappings: L3 Network = Routing, Packet, IPv4/IPv6/ICMP/IPsec, router/L3 switch; L2 Data Link = Framing, Frame, Ethernet (802.3)/Wi-Fi (802.11)/802.1Q/ARP, switch/AP/bridge/NIC; L1 Physical = Bits, Bit, RF/fiber/copper, cable/radio/hub.

FIELD NOTES

- What it shows: the 7 layers, top (7 Application) to bottom (1 Physical), each with: layer number (lime index), name, a one-word function keyword, PDU (Data/Segment/Packet/Frame/Bit), example protocols, and typical hardware.
- Footnote: PDU = protocol data unit; layers 5 to 7 are commonly grouped as "data" in TCP/IP practice; ARP is widely placed at Layer 2 (some texts call it L2/L3), and it resolves L3 addresses to L2 addresses.
- The function column is a neutral keyword (Keith's decision: no custom mnemonic). Standard reference, not region-specific.
- Data source / standard: ISO/IEC 7498-1:1994 plus standard IETF/IEEE protocol-to-layer mappings; from the Pax research deliverable (pax-research-7-additions.md).

Source: `lib/screens/tools/reference/osi_model_screen.dart`

PLMN ID Reference

An offline, searchable, grouped lookup of US Public Land Mobile Network identifiers. Each row pairs a carrier or operator with its MCC, MNC, full PLMN ID, and operational status, covering MCCs 310-316 (US mainland plus Puerto Rico, Guam, the US Virgin Islands, and American Samoa).

WHY IT'S HERE

When you have a PLMN ID, MCC, or MNC off a cellular scan, a SIM, or a private-LTE / CBRS deployment and need to know which carrier it belongs to, or you need a carrier's code to configure or verify a private cellular network. It works fully offline, so it is reliable in the field where there is no data connection.

HOW TO USE

1. Browse the entries grouped by MCC (310 through 316), each group sorted ascending by PLMN ID.
2. Type in the search box to filter live by code (MCC, MNC, or full PLMN ID) or by carrier / operator name.
3. Each row shows the PLMN ID, the MCC/MNC pair, the carrier (and parent operator when different), and the operational status.
4. Use the copy action to grab the current view (the filtered subset when searching, otherwise the full table) as plain text.

FIELD NOTES

- MNC and PLMN ID are strings with significant leading zeros (e.g. MNC 004, PLMN ID 310004). A two-digit and a three-digit MNC are different codes; never read them as numbers.
- Status values include operational, not operational, reserved, and unknown. A reserved or not-operational allocation may still appear on the air or in old records, so the status column matters.
- US-only by design. The dataset covers ITU region 3xx (MCCs 310-316); it does not include non-US carriers.
- Data source: US MCC/MNC (PLMN ID) allocations verified against the live Wikipedia 'Mobile country code' tables (2026-06-05). The bundled JSON is the source of truth and can be edited to add or correct entries.

Source: `lib/screens/tools/reference/plmn_reference_screen.dart`

Top-Level Domains

A curated reference to the DNS top-level domains a network or IT pro actually meets, grouped by registry type: generic (gTLD), country-code (ccTLD), sponsored/restricted, infrastructure, and notable newer gTLDs. Each entry shows the TLD, its type, and a short managed-by / typical-use note.

WHY IT'S HERE

Knowing a TLD's type tells you who runs it and what to expect: a sponsored domain like .gov or .edu is eligibility-verified, .arpa is reverse-DNS infrastructure you never register, and .io or .ai are country-code domains used generically rather than true gTLDs.

HOW TO USE

1. Scroll the grouped cards, or use the Type filter to narrow to one registry class.
2. Read the TLD (lime, left) then its note. Copy exports the full curated set as a table, regardless of the active filter.

FIELD NOTES

- Curated, not exhaustive: the live root zone has roughly 1,500 generic TLDs and about 250 country-code TLDs. This lists the meaningful, field-relevant set.
- Accuracy: .io (British Indian Ocean Territory), .ai (Anguilla), .co (Colombia), .tv and .me are technically country-code TLDs commonly used generically. They are NOT true generic TLDs, and the notes say so.
- Data source / standard: the IANA Root Zone Database for registry classification and sponsoring organizations; ICANN for new-gTLD program facts.

Source: `lib/screens/tools/reference/top_level_domains_screen.dart`

Well-Known Ports

Searchable, offline reference of 86 curated TCP/UDP ports a network or Wi-Fi pro meets in the field. Search by port number or by service-name / description substring.

WHY IT'S HERE

At a packet capture or writing a firewall rule, you ask two questions: "what runs on port N?" and "what port does service X use?" This answers both without leaving the app or going online.

HOW TO USE

1. Type a port number (e.g. 443) for an exact-port lookup, or a service name / keyword (e.g. radius, dns, vpn) for a case-insensitive substring match against both the service name and the description.
2. An empty search box lists all 86 curated ports, sorted ascending by port number.
3. Each result shows the service name, the port number with its protocol label (TCP, UDP, or both), and a one-line description.
4. A query that matches nothing shows an honest "No match" card; it never fabricates a port.

FIELD NOTES

- What it contains: 86 curated entries spanning ports 1 to 27017 (e.g. 53 dns TCP/UDP, 67 dhcp UDP, 123 ntp UDP, 443 https TCP/UDP where UDP/443 carries HTTP/3 QUIC, 1812 radius UDP and 1813 radius-acct UDP for 802.1X / WPA2-Enterprise, 3389 rdp). Each entry: port number, protocol(s), short service name, one-line description.
- Protocols in the table are TCP and/or UDP only (no SCTP entries present, though the schema supports it). Combinations: 38 TCP-only, 26 UDP-only, 22 TCP+UDP.
- This is a curated subset of the IANA Service Name and Transport Protocol Port Number Registry, trimmed to field-relevant ports, not the full ~49,000-entry registry. Absence means "not in our curated set," which the screen states plainly rather than inventing a row.
- Fully offline: the table is a bundled asset (assets/ports/well_known_ports.json) loaded and indexed once at startup; numeric lookups are O(1) via a port index. Works on every platform.
- To add or correct an entry, edit the asset file; PortReferenceService re-indexes it at load. Malformed rows are skipped, not rendered as blanks.

Source: `lib/screens/tools/network/port_reference_screen.dart`

Encoding 2

ASCII / Hex / Binary

The full 128-character US-ASCII table with decimal, hex, octal, and binary for each code, plus supplementary quick-reference tables for reading hex dumps and protocol fields.

WHY IT'S HERE

When decoding a hex dump or a protocol field, look up a byte's character, its four numeric representations, or the meaning of a control code.

HOW TO USE

1. The numeric columns (dec/hex/oct/bin) and the glyph render in Roboto Mono so octets and look-alike characters (l/I/O/0) read unambiguously.
2. Control rows show a mnemonic (NUL, LF, CR, ESC…); printable rows show the glyph (space is shown as "SP").
3. Filter by typing a decimal, hex (with or without 0x), octal, binary, glyph/mnemonic, or keyword.

FIELD NOTES

- What it shows: a "how to read this" card, then Control codes (0 to 31, plus 127) and Printable characters (32 to 126) as tables with Dec / Hex / Oct / Bin / Char / Description. Plus supplementary cards: Range boundaries worth memorizing, Newlines on the wire, The case bit (0x20), Nibble → hex map, Powers of two, Hex place values, and High range (128 to 255): no single "extended ASCII".
- The high-range card is explicitly honest: ASCII stops at 127; bytes 128 to 255 mean different things depending on the encoding, so there is no single "extended ASCII." It documents UTF-8 (bytes 0 to 127 identical to ASCII; 128 to 255 are part of multi-byte sequences), ISO-8859-1/Latin-1, and Windows-1252 (a common mojibake source), with the rule "bytes 0 to 127 are portable; 128 to 255 are not, so know the encoding before decoding."
- Data source / standard: RFC 20 (US-ASCII) for the 128 values; high-range guidance per ISO-8859-1, Windows-1252, and the Unicode/UTF-8 spec. Embedded verbatim from Deliverables/2026-05-31-ascii-hex-binary-reference/. Standard reference, not region-specific.

Source: `lib/screens/tools/reference/ascii_reference_screen.dart`

Top 30 Emoji

The 30 most-used emoji, ranked 1 to 30, with the official Unicode CLDR name and a descriptive "common use" note.

WHY IT'S HERE

A lightweight decode reference for what people actually mean by an emoji, useful when meanings drift (e.g. 💀 = "that's hilarious," not death).

HOW TO USE

1. Ranked most-used first (😂 face with tears of joy at #1).
2. The official CLDR name is the row's spoken/searchable key; the glyph is excluded from the screen-reader label (readers announce the glyph's own name).
3. The "common use" note describes typical reading, with generational/fandom context where it matters.

FIELD NOTES

- What it shows: per emoji: rank (1 to 30), the glyph, the Unicode CLDR official name, and a "common use" note on how people usually read it today.
- The intro is explicit: "common use" is how people usually read each emoji today, not an official Unicode definition; meanings drift by audience, region, and generation.
- The dataset's literal/codepoint fields are intentionally omitted (Keith's instruction). The glyph renders via the platform color-emoji font (Apple Color Emoji on iOS/macOS).
- Data source / standard: Unicode CLDR for the names; ranking is messaging-weighted (private-messaging keyboard frequency, Keith's decision 2026-05-31, not social-listening-weighted). Embedded verbatim from Deliverables/2026-06-01-emoji-top-30/emoji-top-30.json.

Source: `lib/screens/tools/reference/emoji_reference_screen.dart`

CLI & Capture 3

Linux / WLAN Commands

A grouped Linux command reference for WLAN work: file/process basics, modern and legacy networking, the wireless-specific tools (iw, iwconfig, airmon-ng, rfkill), tested monitor-mode sequences, and the macOS non-root packet-capture setup.

WHY IT'S HERE

You're driving a WLAN Pi, a Linux capture box, or a survey laptop and need to put an adapter into monitor mode on a specific channel/width, or recall the exact iw syntax to read the current link.

HOW TO USE

1. Commands are grouped (File, Directory, Process, Network, Wireless, Monitor-mode, macOS capture).
2. Filter by command or group name; a group-label match surfaces the whole group.

EXAMPLE

> Commands as shipped. Wireless: iw dev, iw dev wlan0 info (type/channel/mode), iw dev wlan0 link (SSID/signal/rate), iw dev wlan0 scan (sudo), iw dev wlan0 set channel 6, iw phy (PHY caps), iwconfig (legacy), iwlist wlan0 scan (legacy), rfkill list, rfkill unblock wifi. Monitor-mode: sudo airmon-ng start wlan0 (creates wlan0mon), sudo airmon-ng start wlan0 36 (monitor + channel 36), sudo airmon-ng stop wlan0mon, sudo airmon-ng check kill, sudo ifconfig wlan0 down / sudo iwconfig wlan0 mode monitor / sudo ifconfig wlan0 up (3-step), sudo iw dev wlan0 set channel 36 HT40+ (40 MHz secondary above), sudo iw dev wlan0 set channel 40 HT40- (40 MHz secondary below), sudo iwconfig wlan0 mode managed, sudo iw dev wlan0 info, lsusb, sudo dmesg, sudo ethtool -i wlan0, lsmod. macOS capture: sudo dseditgroup -o edit -a USERNAME -t user access_bpf (non-root capture), dscl . read /Groups/access_bpf (verify membership), sudo wdutil info (macOS 14+).

FIELD NOTES

- In-app caveat: iwconfig/iwlist/ifconfig are legacy "wireless extensions" tools; modern distros prefer iw and the iproute2 suite; monitor-mode sequences need sudo and a capable adapter/driver.
- In-app footnote: wireless extensions are deprecated in favor of iw + nl80211 (both shown because field gear still ships the legacy tools); HT40+/HT40- selects a 40 MHz channel with the secondary 20 MHz channel above (+) or below (-) the control channel; the access_bpf group lets a non-root macOS user capture via libpcap (BPF devices). Replace USERNAME and wlan0 with your actual user and interface.
- Source / basis: targets Linux primarily, with a macOS-capture group. Dataset is the Pax research deliverable (pax-research-7-additions.md, "Linux / WLAN Commands"), cross-checked against Linux man-pages, iw/nl80211 docs, and aircrack-ng docs.

Source: `lib/screens/tools/command/linux_wlan_commands_screen.dart`

Network CLI Commands

A side-by-side Windows vs macOS/Linux command reference for the everyday network-troubleshooting tasks (reachability, path tracing, DNS, interface config, sockets, ARP, routing, Wi-Fi link state), each with the field-common flags.

WHY IT'S HERE

You're at a client site on whatever laptop is in front of you and need the right command for this OS without looking it up: "what's the macOS equivalent of ipconfig /all," "how do I see the connected SSID/BSSID/RSSI from the CLI on Windows."

HOW TO USE

1. One card per task. Each card shows the Windows command (lime), the macOS/Linux equivalent, a one-line description, and a flag subset.
2. Filter by command name or task (e.g. "ping" or "DNS"). Where a platform has no native command, the card says so honestly rather than blanking.

EXAMPLE

> Commands as shipped (Task | Windows | macOS/Linux | key flags): Reachability/RTT via ICMP echo | ping host | ping host | -t (Win: continuous), -n count (Win), -l size (Win), -c count (nix), -s size (nix), -i interval (nix). Trace L3 path | tracert host | traceroute host | -d (Win: no resolve), -h max (Win), -n (nix), -m max (nix), -I (nix: ICMP), -T (Linux: TCP SYN), -P proto (macOS). Ping+traceroute over time | pathping host | mtr host | -n, -q num (pathping). DNS query legacy | nslookup name | nslookup name | -type=MX, server. DNS query full detail | (no native command) | dig name | +short, -x addr, @server, type. Interface IP config | ipconfig /all | ifconfig | /all, /release, /renew, /flushdns, ip addr (Linux), ipconfig getifaddr en0 (macOS). Active connections | netstat -ano | netstat -an | -a, -n, -o (Win PID), -r, -p proto. ARP cache | arp -a | arp -a | -a, -d addr, -s addr mac. Device hostname | hostname | hostname | -f (nix FQDN). IP routing table | route print | netstat -rn | print (Win), -rn (nix), ip route (Linux). NetBIOS-over-TCP name table | nbtstat -A addr | (no native command) | -A addr, -n. Connected Wi-Fi interface state | netsh wlan show interfaces | wdutil info | show interfaces (Win), show profiles, show networks mode=bssid, show wlanreport, show drivers, wdutil info (macOS, sudo).

FIELD NOTES

- The in-app caveat warns that some commands need administrator/sudo rights and that the flags shown are the field-common subset, not exhaustive.
- The in-app footnote notes that ifconfig, route, and arp are legacy on Linux (modern distros prefer the iproute2 suite: ip addr, ip route, ip neigh); on macOS use wdutil info (sudo) or the Wireless Diagnostics app for Wi-Fi link details; and netsh wlan is Windows only.
- Source / basis: targets Windows and macOS/Linux. Dataset is the Pax research deliverable (pax-research-7-additions.md, "Network CLI Commands"), sourced from Linux man-pages, Microsoft Learn, and Apple docs. Per a Keith decision (2026-05-30) the macOS Wi-Fi entry shows only wdutil info; the deprecated airport CLI was removed entirely.

Source: `lib/screens/tools/command/cli_commands_screen.dart`

Wireshark 802.11 Filters

Copy-ready Wireshark display filters (typed into the filter bar after capture) and capture filters (BPF syntax, applied during capture) for 802.11 analysis: frame type/subtype, addressing, BSSID/SSID, RadioTap metadata, and RSN cipher/AKM selectors.

WHY IT'S HERE

You have a capture open and need the exact display-filter field to isolate deauths, beacons, a specific BSSID, or a security cipher, without guessing field names from memory.

HOW TO USE

1. Filters are grouped; filter the list by syntax or task, and a group-label match surfaces the whole group.
2. The syntax is selectable for copy.

EXAMPLE

> Filters as shipped. Frame type/subtype (display): wlan.fc.type == 0 (all management), == 1 (all control), == 2 (all data); wlan.fc.type_subtype == 0 (Assoc req), 1 (Assoc resp), 2 (Reassoc req), 3 (Reassoc resp), 4 (Probe req), 5 (Probe resp), 8 (Beacon), 9 (ATIM), 10 (Disassoc), 11 (Auth), 12 (Deauth), 13 (Action), 24 (Block Ack Req), 25 (Block Ack), 26 (PS-Poll), 27 (RTS), 28 (CTS), 29 (Ack), 36 (Null data), 40 (QoS data), 44 (QoS Null). Address (display): wlan.addr == aa:bb:cc:dd:ee:ff (any field), wlan.ta, wlan.ra, wlan.sa, wlan.da. BSSID/SSID: wlan.bssid == ..., wlan.ssid == "MyNetwork", wlan.ssid contains "Guest". RadioTap: radiotap.channel.freq == 2412, radiotap.datarate >= 6, radiotap.dbm_antsignal > -70, radiotap.dbm_antnoise < -90, radiotap.channel.freq >= 2400 && < 2500 (2.4 GHz), >= 5000 && < 5900 (5 GHz), >= 5925 && <= 7125 (6 GHz). Capture filter (BPF): type mgt, type ctl, type data, type mgt subtype beacon, type mgt subtype probe-req, type mgt subtype deauth, type ctl subtype rts, type ctl subtype ack, wlan host aa:bb:cc:dd:ee:ff. RSN cipher (display): wlan.rsn.pcs.type == 4 (CCMP-128, 00-0F-AC:4), == 8 (GCMP-128, 00-0F-AC:8), == 9 (GCMP-256, 00-0F-AC:9), wlan.rsn.gcs.type == 2 (group cipher TKIP, 00-0F-AC:2). RSN AKM (display): wlan.rsn.akms.type == 1 (802.1X, 00-0F-AC:1), == 2 (PSK, 00-0F-AC:2), == 8 (SAE / WPA3-Personal, 00-0F-AC:8), == 18 (OWE, 00-0F-AC:18).

FIELD NOTES

- In-app caveat: display-filter field names match Wireshark's dfref; capture filters use libpcap/BPF "type/subtype" syntax and only work when capturing with a RadioTap/PPI header.
- In-app footnote: type_subtype is the combined value (type in the high bits, subtype in the low bits) matching IEEE 802.11 frame type/subtype assignments; capture filters require capturing with a RadioTap header (monitor mode); for the full RSN cipher/AKM number-to-name map, see the RSN groups or the WPA Security reference tool.
- FLAGGED (intentional, not a defect): two deliberate corrections are baked in from the source code header. (1) The RSN cipher-suite vs AKM tables were rebuilt from IEEE 802.11-2020 Tables 9-149 (cipher = wlan.rsn.pcs.type / wlan.rsn.gcs.type) and 9-151 (AKM = wlan.rsn.akms.type) because the original source card mislabeled cipher values as AKM. (2) The 5 GHz/2.4 GHz/6 GHz band filters ship a deliberate SAFE FALLBACK using documented radiotap.channel.freq ranges instead of the unverified radiotap.channel.flags.5ghz child-token. Band-edge detail: the 5 GHz range stops at < 5900 and the 6 GHz range starts at >= 5925, so center frequencies in the 5900 to 5924 MHz gap fall into neither band filter. This matches the shipped code exactly and is intentional.
- Source / basis: targets Wireshark display-filter (dfref) and libpcap/BPF capture-filter conventions. Dataset is the Pax research deliverable (pax-research-7-additions.md, "Wireshark 802.11 Filters"), sourced from the Wireshark dfref, the RadioTap dfref, pcap-filter(7), and IEEE 802.11-2020.

Source: `lib/screens/tools/command/wireshark_filters_screen.dart`

Checklists 2

How to NOT Have a Wireless Problem

A run-through-it AP install checklist organized into Before / Install / After phases, most of which is wired-side verification, on the premise that many "wireless" problems are not wireless problems.

WHY IT'S HERE

You're mounting and turning up an access point and want to confirm the cabling, PoE, VLAN, DHCP, and routing path are right before you blame the radio, then document and validate after.

HOW TO USE

1. Three phases; tap each item as you complete it; the top count tracks progress.
2. Intro line in the app: many wireless problems are not wireless problems. Work these checks before/during/after the install, using a LinkSprinter, LinkRunner AT, EtherScope, or CyberScope, or just a laptop with a command window and the right commands.

EXAMPLE

> Before Installing Access Point: Cable meets/exceeds Cat5e specs · Total cable distance with patch cords < 100 m · PoE meets the AP's specific requirements · Check 802.3 af, at, or bt · Confirm DHCP address & VLAN · Confirm correct VLAN assignment · Confirm access or trunk port as required · Confirm default gateway · Ping default gateway · Confirm target IP addresses reachable · Confirm DNS reachable · Confirm target DNS addresses reachable · Management VLAN assigned & available. Install Access Point: Install access point (kept as its own one-item phase). After Installing Access Point: Document AP's MAC & assigned name · Document AP's location · Document AP's switch / port used · Document AP's IP address · Confirm AP installed in proper orientation · Confirm external antennas installed correctly · Wait for access point to receive configuration · Wait for a 2nd reboot of the AP if needed · Listen in air for all SSIDs being broadcast · Connect client to each SSID · Check each SSID for proper VLAN & IP pool.

FIELD NOTES

- Checked state is not persisted; it resets when you leave the screen. The tool wires the install card's content into the shared checklist screen via kApInstallChecklist.
- Source / basis: Keith Parsons / WLAN Pros original card (© 2024 WLAN Pros), transcribed by Pax (pax-research-7-additions.md) with obvious typos fixed. Per a Keith decision (2026-05-30), the "After Installing" list is renumbered to a clean 1-12 (the original card was gap-numbered with no item 2); ChecklistScreen numbers by render order.

Source: `lib/data/checklists.dart (content) / lib/screens/tools/checklists/checklist_screen.dart (screen type)`

Wi-Fi Client Testing Checklist

An ordered, twelve-step client-side connectivity test to run from a client device after an install or when triaging a connectivity complaint.

WHY IT'S HERE

A user reports "Wi-Fi is broken." This walks you from "can the client even see the SSID" through association, auth, DHCP, gateway/DNS reachability, data rate, and a speed test, in the order failures actually cascade.

HOW TO USE

1. One flat ordered list (no phases). Tap each as you complete it; the count tracks progress.

EXAMPLE

> Items in order: Can see all SSIDs being broadcast · Associate to target SSID · Complete SSID authentication · Receive an IP address via DHCP · Receive default gateway & DNS · Ping default gateway · Ping DNS · Ping remote IP address · Ping remote DNS address · Check client MCS · Check client Tx data rate · Complete network speed test.

FIELD NOTES

- Per-session only (not saved). Wired into the shared screen via kClientTestChecklist.
- Source / basis: Keith Parsons / WLAN Pros original card (© 2024 WLAN Pros), transcribed by Pax (pax-research-7-additions.md).

Source: `lib/data/checklists.dart (content) / lib/screens/tools/checklists/checklist_screen.dart (screen type)`

Guides 2

Dual Orbs on WLAN Pi

A step-by-step how-to that turns a WLAN Pi R4 or M4+ into two Orb sensors (one testing the wired connection over Ethernet, one testing Wi-Fi) by installing a bundled Debian package. The screen walks the scp / apt install / reboot flow, covers the cloned-image identity reset, shows how to reconfigure the Wi-Fi credentials, and lists the service-management commands. A Download button exports the bundled wlanpi-dual-orb_1.1.3_all.deb via the share sheet.

WHY IT'S HERE

When you want continuous, two-path connection monitoring (wired and wireless) from one small device you already carry, this gets a WLAN Pi running two free Orb sensors in a few commands. The results show up in your own Orb account; the Toolbox just gives you the guide and the package.

HOW TO USE

1. Tap Download wlanpi-dual-orb.deb and save it (Files / AirDrop / Mail). Keep the filename; the install command refers to it by name.
2. Copy the package to your WLAN Pi: scp wlanpi-dual-orb_1.1.3_all.deb wlanpi@<your-Pi-IP>:~
3. SSH to the Pi and install it: sudo apt install ./wlanpi-dual-orb_1.1.3_all.deb (it prompts for the Wi-Fi SSID, password, and encryption).
4. Reboot: sudo reboot. orb-install.service runs on boot, installs Orb, and starts both sensors.
5. If you cloned the image onto more than one Pi, run sudo orb-reset-identity on each so the Orb dashboard sees a distinct new sensor.
6. To change the Wi-Fi credentials later, run sudo orb-wifi-configure.

FIELD NOTES

- What it installs: the free, open-source Orb sensor on the Pi, with its services running as root. You view results in your own Orb account, free for up to 5 devices. Nothing of Orb's is redistributed by the Toolbox; the bundled package only sets the sensor up.
- Tested on the WLAN Pi R4 and the M4+.
- On cloned images the OrbIDs and hostnames carry over from the original; sudo orb-reset-identity deletes the old ones and derives fresh IDs and names from the WLAN Pi hostname. Use sudo orb-reset-identity --dry-run to preview without changing anything.
- The Wi-Fi sensor runs in its own network namespace (orb-wifi); inspect it with sudo ip netns exec orb-wifi iw dev or sudo ip netns exec orb-wifi ip link show.
- Guide and package by Ferney Munoz. Orb: orb.net. WLAN Pi project: wlanpi.com.

Source: `lib/screens/tools/guides/dual_orb_screen.dart`

FreeRADIUS on WLAN Pi

A step-by-step guide to standing up a lab RADIUS server on a WLAN Pi for learning and testing 802.1X (WPA2/WPA3-Enterprise), built around a bundled install script you copy to the Pi and run there.

WHY IT'S HERE

Reach for it when you want a working RADIUS server to practice 802.1X against without building a full production AAA stack. The screen shows the install script inline, lets you download it, and points at the two files to customize (the shared secret and the user list).

HOW TO USE

1. Read the lab caveat: the script ships test accounts (student01-student10) with cleartext passwords and a shared secret named secretwlanpros. Change the secret and use real credentials before anything real.
2. Download install_freeradius.sh (or copy it from the inline view).
3. Copy it to the Pi: scp install_freeradius.sh wlanpi@<your-Pi-IP>:~
4. Make it executable: chmod +x install_freeradius.sh
5. Run it on the Pi: ./install_freeradius.sh
6. Verify with the listed radtest / journalctl / tcpdump commands.

FIELD NOTES

- The Toolbox runs no shell. This is reference text plus a file you run on your own WLAN Pi, not in the app. The inline script and the download are the exact same bytes.
- Lab use only as shipped: cleartext-password test accounts and a known shared secret. Change the secret in /etc/freeradius/3.0/clients.conf and replace the test users in /etc/freeradius/3.0/users before using it on a real network.
- The script configures PEAP/MSCHAPv2, opens UDP 1812, and runs a radtest against student01 to confirm the server answers.
- Guide and install script by Ferney Munoz; bundled with permission.

Source: `freeradius_wlanpi_screen.dart + assets/downloads/install_freeradius.sh (Ferney Munoz)`

Reference Cards 10

2.4 GHz Channel Allocations

A bundled, offline, pinch-zoomable copy of Keith's published "2.4 GHz channel layout and allocations" laminated reference card.

WHY IT'S HERE

You want the canonical WLAN Pros 2.4 GHz channel allocation map exactly as printed, viewable and zoomable on the phone you already have, with no network needed.

HOW TO USE

1. Open the card and pinch / double-tap to zoom; the page is fit-to-screen on open.
2. This is a flat print PDF, not an in-app data table, so the content is not screen-reader readable.

FIELD NOTES

- Card inner content is not accessible to screen readers (flat rasterized PDF). The card title and "pinch to zoom" gesture are announced.
- Source / basis: Keith's own published WLAN Pros laminated reference card, exported from Excel to PDF, bundled under assets/reference-cards/channel-allocations-24ghz.pdf. Rendered offline via the pdfx PdfView viewer (Apple PDFKit on iOS + macOS).

Source: `lib/screens/tools/reference/pdf_reference_screen.dart (shared viewer) / assets/reference-cards/channel-allocations-24ghz.pdf`

5 GHz Channel Allocations

A bundled, offline, pinch-zoomable copy of Keith's published "5 GHz channel layout and allocations" laminated reference card.

WHY IT'S HERE

You want the canonical WLAN Pros 5 GHz channel allocation map exactly as printed, viewable and zoomable on the phone you already have, with no network needed.

HOW TO USE

1. Open the card and pinch / double-tap to zoom; the page is fit-to-screen on open.
2. This is a flat print PDF, not an in-app data table, so the content is not screen-reader readable.

FIELD NOTES

- Card inner content is not accessible to screen readers (flat rasterized PDF). The card title and "pinch to zoom" gesture are announced.
- Source / basis: Keith's own published WLAN Pros laminated reference card, exported from Excel to PDF, bundled under assets/reference-cards/channel-allocations-5ghz.pdf. Rendered offline via the pdfx PdfView viewer (Apple PDFKit on iOS + macOS).

Source: `lib/screens/tools/reference/pdf_reference_screen.dart (shared viewer) / assets/reference-cards/channel-allocations-5ghz.pdf`

---

6 GHz Channel Allocations

A bundled, offline, pinch-zoomable copy of Keith's published "6 GHz channel layout and allocations" laminated reference card.

WHY IT'S HERE

You want the canonical WLAN Pros 6 GHz channel allocation map exactly as printed, viewable and zoomable on the phone you already have, with no network needed.

HOW TO USE

1. Open the card and pinch / double-tap to zoom; the page is fit-to-screen on open.
2. This is a flat print PDF, not an in-app data table, so the content is not screen-reader readable.

FIELD NOTES

- Card inner content is not accessible to screen readers (flat rasterized PDF). The card title and "pinch to zoom" gesture are announced.
- Source / basis: Keith's own published WLAN Pros laminated reference card, exported from Excel to PDF, bundled under assets/reference-cards/channel-allocations-6ghz.pdf. Rendered offline via the pdfx PdfView viewer (Apple PDFKit on iOS + macOS).

Source: `lib/screens/tools/reference/pdf_reference_screen.dart (shared viewer) / assets/reference-cards/channel-allocations-6ghz.pdf`

Extended Checklist (Non-Advertised Items)

A bundled, offline, pinch-zoomable copy of Keith's published "extended checklist, non-advertised items" laminated reference card.

WHY IT'S HERE

You want the canonical WLAN Pros extended (non-advertised items) checklist card art exactly as printed, viewable and zoomable on the phone you already have, with no network needed.

HOW TO USE

1. Open the card and pinch / double-tap to zoom; the page is fit-to-screen on open.
2. This is a flat print PDF, not an in-app data table, so the content is not screen-reader readable.

FIELD NOTES

- Card inner content is not accessible to screen readers (flat rasterized PDF). The card title and "pinch to zoom" gesture are announced.
- Source / basis: Keith's own published WLAN Pros laminated reference card, exported from Excel to PDF, bundled under assets/reference-cards/extended-checklist-nonadvertised.pdf. Rendered offline via the pdfx PdfView viewer (Apple PDFKit on iOS + macOS).

Source: `lib/screens/tools/reference/pdf_reference_screen.dart (shared viewer) / assets/reference-cards/extended-checklist-nonadvertised.pdf`

Extended Wi-Fi Checklist

A bundled, offline, pinch-zoomable copy of Keith's published "extended design checklist items" laminated reference card.

WHY IT'S HERE

You want the canonical WLAN Pros extended checklist card art exactly as printed, viewable and zoomable on the phone you already have, with no network needed.

HOW TO USE

1. Open the card and pinch / double-tap to zoom; the page is fit-to-screen on open.
2. This is a flat print PDF, not an in-app data table, so the content is not screen-reader readable.

FIELD NOTES

- Card inner content is not accessible to screen readers (flat rasterized PDF). The card title and "pinch to zoom" gesture are announced.
- Source / basis: Keith's own published WLAN Pros laminated reference card, exported from Excel to PDF, bundled under assets/reference-cards/extended-checklist.pdf. Rendered offline via the pdfx PdfView viewer (Apple PDFKit on iOS + macOS).

Source: `lib/screens/tools/reference/pdf_reference_screen.dart (shared viewer) / assets/reference-cards/extended-checklist.pdf`

Modulation and Coding Schemes (MCS Index)

A bundled, offline, pinch-zoomable copy of Keith's published "MCS index, rates, and modulation" laminated reference card.

WHY IT'S HERE

You want the canonical WLAN Pros MCS index card art exactly as printed, viewable and zoomable on the phone you already have, with no network needed.

HOW TO USE

1. Open the card and pinch / double-tap to zoom; the page is fit-to-screen on open.
2. This is a flat print PDF, not an in-app data table, so the content is not screen-reader readable.

FIELD NOTES

- Deliberate id split: mcs-index-card (this PDF card) is separate from the mcs-index interactive data-table tool.
- Card inner content is not accessible to screen readers (flat rasterized PDF). The card title and "pinch to zoom" gesture are announced.
- Source / basis: Keith's own published WLAN Pros laminated reference card, exported from Excel to PDF, bundled under assets/reference-cards/mcs-index-card.pdf. Rendered offline via the pdfx PdfView viewer (Apple PDFKit on iOS + macOS).

Source: `lib/screens/tools/reference/pdf_reference_screen.dart (shared viewer) / assets/reference-cards/mcs-index-card.pdf`

Top 20 Wi-Fi Checklist

A bundled, offline, pinch-zoomable copy of Keith's published "Top 20 Wi-Fi design checklist" laminated reference card (PDF card form).

WHY IT'S HERE

You want the canonical WLAN Pros Top 20 checklist card art exactly as printed, viewable and zoomable on the phone you already have, with no network needed.

HOW TO USE

1. Open the card and pinch / double-tap to zoom; the page is fit-to-screen on open.
2. This is a flat print PDF, not an in-app data table, so the content is not screen-reader readable.

FIELD NOTES

- Deliberate id split: top-20-checklist here is the PDF card (separate from any tappable checklist).
- Card inner content is not accessible to screen readers (flat rasterized PDF). The card title and "pinch to zoom" gesture are announced.
- Source / basis: Keith's own published WLAN Pros laminated reference card, exported from Excel to PDF, bundled under assets/reference-cards/top-20-checklist.pdf. Rendered offline via the pdfx PdfView viewer (Apple PDFKit on iOS + macOS).

Source: `lib/screens/tools/reference/pdf_reference_screen.dart (shared viewer) / assets/reference-cards/top-20-checklist.pdf`

Wi-Fi Connection Checklist

A bundled, offline, pinch-zoomable copy of Keith's published "client connection sequence checklist" laminated reference card.

WHY IT'S HERE

You want the canonical WLAN Pros connection-sequence checklist card art exactly as printed, viewable and zoomable on the phone you already have, with no network needed.

HOW TO USE

1. Open the card and pinch / double-tap to zoom; the page is fit-to-screen on open.
2. This is a flat print PDF, not an in-app data table, so the content is not screen-reader readable.

FIELD NOTES

- Card inner content is not accessible to screen readers (flat rasterized PDF). The card title and "pinch to zoom" gesture are announced.
- Source / basis: Keith's own published WLAN Pros laminated reference card, exported from Excel to PDF, bundled under assets/reference-cards/connection-checklist.pdf. Rendered offline via the pdfx PdfView viewer (Apple PDFKit on iOS + macOS).

Source: `lib/screens/tools/reference/pdf_reference_screen.dart (shared viewer) / assets/reference-cards/connection-checklist.pdf`

Wireless LAN Troubleshooting Causes

A bundled, offline, pinch-zoomable copy of Keith's published "common causes to check when troubleshooting" laminated reference card.

WHY IT'S HERE

You want the canonical WLAN Pros troubleshooting-causes card art exactly as printed, viewable and zoomable on the phone you already have, with no network needed.

HOW TO USE

1. Open the card and pinch / double-tap to zoom; the page is fit-to-screen on open.
2. This is a flat print PDF, not an in-app data table, so the content is not screen-reader readable.

FIELD NOTES

- Ships as a PDF card on purpose: print-layout artwork, distinct from the equivalent in-app data tables.
- Card inner content is not accessible to screen readers (flat rasterized PDF). The card title and "pinch to zoom" gesture are announced.
- Source / basis: Keith's own published WLAN Pros laminated reference card, exported from Excel to PDF, bundled under assets/reference-cards/troubleshooting-causes.pdf. Rendered offline via the pdfx PdfView viewer (Apple PDFKit on iOS + macOS).

Source: `lib/screens/tools/reference/pdf_reference_screen.dart (shared viewer) / assets/reference-cards/troubleshooting-causes.pdf`

WLAN Pros Bubble Diagram

A bundled, offline, pinch-zoomable copy of Keith's published "Wi-Fi design decision bubble diagram" laminated reference card.

WHY IT'S HERE

You want the canonical WLAN Pros bubble-diagram design-decision flow exactly as printed, viewable and zoomable on the phone you already have, with no network needed.

HOW TO USE

1. Open the card and pinch / double-tap to zoom; the page is fit-to-screen on open.
2. This is a flat print PDF, not an in-app data table, so the content is not screen-reader readable (the screen names the card and the gesture as the honest accessible affordance).

FIELD NOTES

- Ships as a PDF card on purpose: print-layout artwork, distinct from the equivalent in-app data tables.
- Card inner content is not accessible to screen readers (flat rasterized PDF). The card title and "pinch to zoom" gesture are announced.
- bubble-diagram.pdf is the only rotated card (a portrait MediaBox with a /Rotate 90 flag). The viewer was specifically built around PdfView (PhotoView-backed) rather than PdfViewPinch so this card paints landscape and undistorted; this is a verified fix (2026-06-01), not an open issue.
- Source / basis: Keith's own published WLAN Pros laminated reference card, exported from Excel to PDF, bundled under assets/reference-cards/bubble-diagram.pdf. Rendered offline via the pdfx PdfView viewer (Apple PDFKit on iOS + macOS).

Source: `lib/screens/tools/reference/pdf_reference_screen.dart (shared viewer) / lib/router/app_router.dart / assets/reference-cards/bubble-diagram.pdf`